Journal of Computational Physics 525 (2025) 113766



Contents lists available at ScienceDirect

Journal of Computational Physics

journal homepage: www.elsevier.com/locate/jcp





p-Multigrid high-order discontinuous Galerkin solver for three-dimensional compressible turbulent flows

D. Bulgarini, A. Ghidoni, E. Mantecca, G. Noventa *

University of Brescia, Department of Mechanical and Industrial Engineering, via Branze 36, Brescia, 25123, Italy

ARTICLE INFO

Keywords:
Discontinuous Galerkin
Multigrid algorithm
RANS equations
Compressible flows

ABSTRACT

The study of turbulent flows through steady-state simulations based on the Reynolds-averaged Navier-Stokes equations and turbulence models can be considered the workhorse in different scientific and industrial applications. Among the different numerical approaches, discontinuous Galerkin methods demonstrated to be perfectly suited for high-order accurate numerical solutions on structured or arbitrary unstructured and non-conforming meshes, and high-performance computing with massively parallel processing. However, their computational cost increases rapidly when the solution is discretized with higher-order polynomial approximations. For this reason, many research efforts have been devoted to overcome this drawback. Literature shows many applications of *p*-multigrid algorithms for the solution of Euler and Navier-Stokes equations, while few works report the solution of the Reynolds-averaged Navier-Stokes equations with *p*-multigrid algorithms. In fact, different authors highlighted a lack of performance for the stiffness associated with the discretized RANS equations, and for highly stretched meshes, typically used for an accurate resolution of turbulent boundary layers. This work presents the implementation of an improved *p*-multigrid algorithm based on the nonlinear full approximation scheme in a discontinuous Galerkin solver for the solution of the three-dimensional and compressible Reynolds-Average Navier-Stokes equations. The performance of the algorithm with different smoothers is compared with the implicit (single-order) time integration on many test cases with different flow conditions, domains, and meshes, showing an average reduction of the computing time around 75% with respect to single-order implicit solvers.

1. Introduction

Nowadays, computational fluid dynamics (CFD) is an important tool for different scientific and industrial applications. Traditional CFD solvers are based on second-order accurate Finite Volume Methods (FVMs), but the need for higher accuracy and the ever-increasing availability of High Performance Computing (HPC) pave the way for the development and use of higher-order accurate methods, such as discontinuous Galerkin (dG). The advantage of dG methods lies in the ability to achieve the same spatial accuracy with a reduced number of degrees of freedom (DOFs) compared to traditional methods, e.g., FV methods. Furthermore, the compact stencil of dG methods, which needs only one element and its neighbors, makes the formulation perfectly suited for Massively Parallel Processing (MPP).

* Corresponding author.
E-mail address: gianmaria.noventa@unibs.it (G. Noventa).

https://doi.org/10.1016/j.jcp.2025.113766
Received 10 July 2024; Received in revised form 10 December 2024; Accepted 19 January 2025

Available online 22 January 2025
0021-9991/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

In dG methods, as in the Finite Element Methods (FEMs), the solution of the weak or variational form of a partial differential problem is approximated by polynomial functions over the elements of a mesh. However, unlike continuous FEMs, the dG method solution approximation is generally discontinuous at element interfaces, where the coupling of the solution between the elements is weakly enforced by numerical flux functions. The drawback of the high-order accuracy, geometric flexibility, and robustness, is the high computational cost and memory footprint. For this reason, significant research efforts have been spent to improve the computational efficiency in the last years. Many solutions have been proposed for the efficient assembly of the discretization operators [1,2], and for the time integration of the discretized governing equations, e.g., the implicit (single-order) time integration [3,4] and the *h*- and *p*-MultiGrid (MG) algorithms [5,6].

Historically, MG has been used directly as nonlinear solvers [7], as linear solvers operating on a linearization arising from a Newton solution strategy of a non-linear problem [8], or as a linear preconditioner for a Newton–Krylov method [9]. MG has been also applied to the high-order framework [10,11]. In general, the main difference between MG as a solver or a preconditioner can be found in the memory usage and convergence speed. In fact, a MG solver can decrease the memory usage, depending on the type of smoother adopted on each level, which can reduce the storage of the Jacobian matrix, e.g., element line Jacobi smoother [10]. On the contrary, a MG preconditioner requires a significant additional memory for the storage of the lower-order Jacobians on each level. To reduce memory usage, the skip of levels has been proposed in literature [12]. In terms of computing time, the MG solver based on smoothers with "reduced Jacobian" shows worse performance compared to the Newton-Krylov method. The use of MG as preconditioner allows a faster convergence of the Newton-Krylov method, but this does not always correspond to a reduction in CPU time compared to standard preconditioners (at least for steady simulations) [13].

The basic idea of the *h*- and *p*-MG algorithms is to use lower-order approximations to correct higher-order approximations, to speed up the convergence of the higher-order approximations. In the *h*-MG algorithm, the discretized governing equations are solved on a series of increasingly coarse meshes, while in the *p*-MG algorithm, on a series of lower-order approximate solutions and the same mesh. The pioneer works of Gopalakrishnan and Kanschat [14], Bastian et al. [15], Antonietti and Melas [16], and Antonietti et al. [17] demonstrated the asymptotic scalability of MG algorithms for dG solvers of Partial Differential Equations (PDEs). The authors demonstrated that MG algorithms are uniformly convergent with respect to the discretization parameters, e.g., the dimensions of the elements of the mesh and the order of the solution. Helenbrook et al. [18] and Fidkowski et al. [19] proposed *p*-MG algorithms for one- and two-dimensional compressible Euler equations, Ghidoni et al. [20,21] and Rueda-Ramírez et al. [22] for the two- and three-dimensional Navier-Stokes (NS) equations, and Ghidoni et al. [23], Colombo et al. [6], Wallraff et al. [24], and Jiang et al. [25] for the Reynolds-Average Navier-Stokes (RANS) equations.

In the MG algorithm, the smoother or relaxation scheme at an arbitrary level, l, must damp all the error modes that cannot be represented on "coarser" levels, where "coarse" can be a coarser mesh or a lower solution approximation. A possible advantage of the *p*- over the *h*-MG algorithm is the easier implementation that avoids the use of different meshes. Different paths can be used to move between the levels during the convergence process (the most popular paths in literature are V- and W-cycle). At each level, a number of pre-/post-smoothing (ν_{pre}/ν_{post}) iterations allow restricting/prolongating the solution to the next coarser/finer level.

Many examples can be found in literature regarding the application of *h*- and *p*-MG algorithms for the solution of Euler and Navier-Stokes (NS) equations, while few examples describe the application of MG algorithms for the solution of three-dimensional RANS equations. Ghidoni et al. [23] spotlighted the reduction of the performance of MG algorithms designed for NS equations, when applied to RANS equations. The reduction can be ascribed to the solution approximation of the coarsest level of the algorithm. In fact, $\mathbb{P}^1$ solution as coarsest level guarantees a lower computational time than $\mathbb{P}^0$ and the corresponding implicit (single-order) simulation. The authors considered benchmark cases as the turbulent flow around the NASA 65° sweep VFE-2 Delta Wing (DW). Wallraff et al. [24] explained this behavior with the stiffness introduced by the RANS equations, and, in particular, by the turbulence model, and with the highly stretched meshes used for an accurate resolution of turbulent boundary layers.

Colombo et al. [6] extended the application of the *p*-MG algorithm to the computation of the turbulent flow through the quasi-three-dimensional and three-dimensional Low-Pressure Turbine (LPT) cascade T106A, designed by MTU Aero Engines, and around a simplified single train coach. Also in this work, the coarsest level of the *p*-MG algorithm was the $\mathbb{P}^1$ solution to guarantee the most favorable conditions for the algorithm. Jiang et al. [25] compared a linear and a nonlinear formulation of the *p*-MG algorithm for turbulent flows, both for steady and unsteady simulations. For the steady-state simulations, the authors proposed an alternative "CFL law", which is well-known to be an important ingredient for an efficient and robust convergence of implicit time integration. In particular, the authors highlighted that the "CFL law" can take advantage of the information of the number of cycles. The results of the simulations of the turbulent flows over a flat plate and around the RAE2822 airfoil and the high-lift multi-element 30P30N airfoil proved the influence of the Courant-Friedrichs-Lewy (CFL) number on the computational cost, while the comparison between the computational cost of the *p*-MG algorithm and the implicit (single-order) time integration is poor, and lacks of detail.

Wallraff et al. [24] compared the computational cost of the *h*- and *p*-MG algorithms with both a linear and nonlinear formulation for the RANS equations and the k-ω and Spalart-Allmaras (SA) turbulence models. The considered test cases are the turbulent flow around the NASA 65° sweep VFE-2 DW and the high-lift multi-element 30P30N airfoil. Also in this work, the authors proposed an alternative "CFL law", the so-called Switched Evolution Relaxation (SER) from Mulder and Van Leer [26]. In the 30P30N case, the linear and nonlinear formulations of the *p*-MG algorithm allowed a CPU time reduction with respect to the implicit (single-order) time integration of 22% and 28%, respectively. The results of the simulations proved that nonlinear formulations are advantageous at the beginning of the simulations at each level, while linear formulations are advantageous at the end. For this reason, according to the authors, the combination of a nonlinear and linear formulation should provide the highest performance for both the *h*- and *p*-MG algorithms.

In this work, the three-dimensional p-MG algorithm (FMG) based on the nonlinear Full Approximation Scheme (FAS) is improved and assessed for the high-order dG approximation of the RANS and the k-$\widetilde{\omega}$ turbulence model equations. Traditional algorithms are characterized by a lot of user-defined parameters, and the choice of these parameters is crucial for the robustness and/or the computational cost. Furthermore, traditional algorithms showed high performance for laminar flows and turbulent flows only when the $\mathbb{P}^1$ solution is used as the coarsest level. In this work, the algorithm is automated to avoid the setting by the user of parameters and tolerances, and other modifications allow to increase the computational efficiency and ease of use, even when the $\mathbb{P}^0$ solution is used as the coarsest level. In particular, the novelty of the algorithm has to be found in:

(i) the logarithmic primitive variables, e.g., $\tilde{p}$ and $\tilde{T}$ and $\widetilde{\omega}$, to increase the smoothness of the solution, and, as a consequence, the robustness of transonic flows simulations [27], where this is of paramount importance in a MG solver. To authors knowledge this is the first time this approach is exploited for a p-MG dG solver (see Sec. 3),
(ii) automatic setting of the MG parameters and tolerances to enhance the robustness and efficiency of the algorithm, where a modified version of the residual-based criterion of Rueda-Ramirez et al. [22] is used (see Sec. 4.4),
(iii) an unbalanced "CFL law" on different levels to force a fast convergence on coarser levels, and to enhance the efficiency (see Sec. 3),
(iv) different parameters and tolerances at each coarser level for the "matrix-explicit" Generalized Minimal RESidual (GMRES) algorithm to reduce the computational cost on coarser levels, where the solution is used to compute the correction term (see Sec. 3),
(v) residuals and Jacobian matrix of the implicit discretization associated with the turbulence model that are computed only on the finest level, and projected on the coarser levels to enhance robustness and efficiency. To authors knowledge the treatment of the turbulence source term is proposed for the first time in a high-order framework (see Sec. 4.2),
(vi) the "skip conditions" to avoid the use of not accurate coarse grid corrections to correct the fine solution (see Sec. 4.4).

The performance of the algorithms with different smoothers, e.g., (i) Block Implicit (BIRK), (ii) Line Implicit Runge-Kutta (LIRK), and (iii) Linearized Backward Euler (LBE) schemes, is compared with the implicit single-order (SO) time integration on different benchmark, verification, and industrial test cases with different flow conditions, e.g., compressible or incompressible and subsonic or transonic, computational domains, e.g., quasi-three and three-dimensional, and meshes with different number and Aspect Ratio (AR) of the elements. The comparison between implicit SO and MG solver is presented considering only a matrix-based strategy. However, a matrix-free (MF) approach [28] could be used for both SO and MG approaches. For a SO computation the MF approach could save at least one matrix storage. Moreover, unlike the matrix-based Newton-Krylov method, the MF is more flexible, as it allows to approximate the preconditioner, e.g., freeze the evaluation of the preconditioner for several Newton steps and/or use an inexact Jacobian matrix, without affecting the quadratic convergence of the Newton algorithm. The memory saving scales with the polynomial order, and, for example, the reduction can be up to 36% for a $\mathbb{P}^6$ solution approximation, as shown in [29]. For a MG computation, the MF approach is always applied on the finest level, to avoid the Jacobian matrix assembly (the preconditioner is still computed and stored), while on the coarser levels different strategies can be adopted: i) project the Jacobian matrix from the finest level, ii) reconstruct the Jacobian matrix on coarser levels, and iii) use a MF approach also on coarser levels.

All the simulations are initialized with free-stream conditions, i.e., the $\mathbb{P}^0$ solution is the coarsest level, which is the standard for industrial applications. Simulations are also initialized with the $\mathbb{P}^1$ solution, as in Ghidoni et al. [23] and Colombo et al. [6], to compare the performance of the smoothers. The transonic turbulent cases are characterized by a sharp solution in the shock flow region, and represent the worst condition to test the performance of h- and p-MG algorithms, as it is well known MG performance deteriorates for the presence of shocks or high AR of the mesh [5,24].

Section 2 describes the RANS governing equations and the k-$\widetilde{\omega}$ turbulence model, while Sec. 3 shows a brief description of the dG numerical framework, focusing on the CFL law and GMRES settings for the multigrid algorithm. Sec. 4 is on the FMG algorithm, while Sec. 5 describes the smoothers compared in this work. After these theoretical descriptions, Sec. 6 compares the proposed smoothers on benchmark testcases in terms of CPU time and memory footprint with respect to a SO strategy, considering a manual and an automatic choice of the smoothing iterations. Section 7 assesses only the performance of the LBE smoother on verification cases for subsonic and transonic regimes, demonstrating the effectiveness of the algorithm for the automatic choice of the smoothing iterations. Finally, Sec. 8 compares the performance of the FMG algorithm with LBE smoother and automatic choice of the smoothing iterations with a SO strategy on complex industrial testcases.

2. RANS and k-$\widetilde{\omega}$ turbulence model

The RANS and $k-\widetilde{\omega}$ turbulence model equations [6] can be written as

$$\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x_j}\left(\rho u_j\right) = 0 \tag{1}$$

$$\frac{\partial}{\partial t}\left(\rho E\right) + \frac{\partial}{\partial x_j}\left(\rho u_j H\right) = \frac{\partial}{\partial x_j}\left[u_i \widehat{\tau}_{ij} - \widehat{q}_j\right] - \tau_{ij}\frac{\partial u_i}{\partial x_j} + \beta^* \rho \overline{k} e^{\widetilde{\omega}_r} \tag{2}$$

$$\frac{\partial}{\partial t}\left(\rho u_i\right) + \frac{\partial}{\partial x_j}\left(\rho u_j u_i\right) = -\frac{\partial p}{\partial x_i} + \frac{\partial \widehat{\tau}_{ij}}{\partial x_j} \tag{3}$$

$$\frac{\partial}{\partial t}(\rho k)+\frac{\partial}{\partial x_j}\left(\rho u_j k\right)=\tau_{ij}\frac{\partial u_i}{\partial x_j}-\beta^*\rho\overline{k}e^{\tilde{\omega}_r}+\frac{\partial}{\partial x_j}\left[\left(\mu+\sigma^*\mu_t\right)\frac{\partial k}{\partial x_j}\right] \tag{4}$$

$$\begin{aligned}\frac{\partial}{\partial t}(\rho\tilde{\omega})+\frac{\partial}{\partial x_j}\left(\rho u_j\tilde{\omega}\right)=&\frac{\alpha}{\overline{k}}\tau_{ij}\frac{\partial u_i}{\partial x_j}-\beta\rho e^{\tilde{\omega}_r}+\frac{\partial}{\partial x_j}\left[\left(\mu+\sigma\mu_t\right)\frac{\partial\tilde{\omega}}{\partial x_j}\right]+\\&+\left(\mu+\sigma\mu_t\right)\frac{\partial\tilde{\omega}}{\partial x_k}\frac{\partial\tilde{\omega}}{\partial x_k}+\sigma_d\rho e^{-\tilde{\omega}_r}\max\left(\frac{\partial k}{\partial x_k}\frac{\partial\tilde{\omega}}{\partial x_k},0\right),\end{aligned} \tag{5}$$

where u_i is the flow velocity, p the pressure, ρ and μ the density and viscosity, E and H the total mass-specific internal energy and enthalpy, τ_{ij} and $\hat{\tau}_{ij}$ the turbulent and the total shear stress tensors, $\hat{q}_j$ is the Fourier's conductive heat flux, μ_t the turbulent viscosity, k the turbulent kinetic energy, and $\tilde{\omega}=\log(\omega)$ the logarithm of the specific dissipation rate, ω. This guarantees the positiveness and smoothness of the turbulent variable in order to increase the robustness of the convergence, which is important for high-order solvers and h- or p-MG algorithms. The different terms are

$$\begin{aligned}&E=\hat{e}+u_k u_k/2,\quad H=h+u_k u_k/2,\quad p=(\gamma-1)\rho(E-u_k u_k/2),\\&\tau_{ij}=2\mu_t\left(S_{ij}-\frac{1}{3}\frac{\partial u_k}{\partial x_k}\delta_{ij}\right)-\frac{2}{3}\rho\overline{k}\delta_{ij},\quad \hat{\tau}_{ij}=2\mu\left(S_{ij}-\frac{1}{3}\frac{\partial u_k}{\partial x_k}\delta_{ij}\right)+\tau_{ij},\\&\hat{q}_j=-\left(\frac{\mu}{Pr}\frac{\mu_t}{Pr_t}\right)\frac{\partial h}{\partial x_j},\quad \mu_t=\alpha^*\rho\overline{k}e^{-\tilde{\omega}_r},\quad \overline{k}=\max(0,k),\end{aligned}$$

where $\gamma=C_p/C_v$ is the ratio of gas specific heat capacities, Pr and Pr_t are the physical and turbulent Prandtl numbers, and

$$S_{ij}=\frac{1}{2}\left(\frac{\partial u_i}{\partial x_j}+\frac{\partial u_j}{\partial x_i}\right),$$

is the strain-rate tensor. The parameters α, α^*, β, β^*, σ, σ^*, and σ_d are those of the high- and low-Reynolds number model of Wilcox [30]. Notice that the turbulent kinetic energy, k, is limited to zero in the turbulent viscosity, μ_t, and in the source terms, and the logarithm of the specific dissipation rate, $\tilde{\omega}$, is used in all the terms instead of the specific dissipation rate, ω, and is limited to $\tilde{\omega}_{r0}$ due to the "realizability" conditions [30], i.e., $\tilde{\omega}_r=\max\left(\tilde{\omega},\tilde{\omega}_{r0}\right)$, in the turbulent viscosity, μ_t, and in the source terms. $\tilde{\omega}_r$ further increases the robustness of the convergence because too small, though positive, values of the logarithm of the specific dissipation rate, $\tilde{\omega}$, can lead to instabilities and breakdown of the simulations. Small values can happen near the convergence through large non-physical variations of the solution, which are characteristic of implicit time integration. The turbulent kinetic energy, k, is not limited in any other term, and can, therefore, be negative in the solution. Negative values can be attributed to poor accuracy and can be removed by mesh refinement and/or higher solution approximations.

3. The numerical framework

The dG discretization of Eqs. (1)-(5) in compact form can be written as

$$\frac{\partial\mathbf{q}}{\partial t}+\boldsymbol{\nabla}\cdot\mathbf{F}^{\mathrm{c}}(\mathbf{q})+\boldsymbol{\nabla}\cdot\mathbf{F}^{\mathrm{v}}(\mathbf{q},\boldsymbol{\nabla}\mathbf{q})+\mathbf{s}(\mathbf{q},\boldsymbol{\nabla}\mathbf{q})=\mathbf{0}\ , \tag{6}$$

where $\mathbf{q}\in\mathbb{R}^m$ is the unknown vector of conservative variables, $\mathbf{F}^{\mathrm{c}}\in\mathbb{R}^m\otimes\mathbb{R}^d$ and $\mathbf{F}^{\mathrm{v}}\in\mathbb{R}^m\otimes\mathbb{R}^d$ are the inviscid and viscous flux functions, $\mathbf{s}\in\mathbb{R}^m$ the source terms of the turbulence model, $m=7$ is the number of variables, and $d=3$ the number of space dimensions. The vector of the primitive variables $\mathbf{w}=[\tilde{p},u_1,u_2,u_3,\tilde{T},k,\tilde{\omega}]^T$ is used instead of the traditional conservative variables, $\mathbf{q}=[\rho,\rho u_1,\rho u_2,\rho u_3,\rho E,\rho k,\rho\tilde{\omega}]^T$. Furthermore, as well as the specific dissipation rate, ω, the logarithms $\tilde{p}=\log(p)$ and $\tilde{T}=\log(T)$ are used instead of the pressure and temperature, p and T. The difference between the use of the logarithm for the turbulent variable and the thermodynamic variables is that the equation for the logarithm of the specific dissipation rate, $\tilde{\omega}$, is explicitly rewritten with an additional source term, while the logarithm of the pressure and temperature, $\tilde{p}$ and $\tilde{T}$, is introduced only in terms of the transformation matrix, $\mathbf{P}$, and in the discretization. These logarithmic variables guarantee the positiveness of the thermodynamic state, even with critical flow phenomena such as strong expansions or shocks, which can introduce instabilities in the convergence. The use of logarithmic variables for both turbulent and thermodynamic variables in a MG framework can lead to a further advantage, as a smoother solution is guaranteed, which is of paramount importance for the robustness of the MG algorithm [5].

Equation (6) can be written for the primitive variables $\mathbf{w}$ as

$$\mathbf{P}(\mathbf{w})\frac{\partial\mathbf{w}}{\partial t}+\boldsymbol{\nabla}\cdot\mathbf{F}^{\mathrm{c}}(\mathbf{w})+\boldsymbol{\nabla}\cdot\mathbf{F}^{\mathrm{v}}(\mathbf{w},\boldsymbol{\nabla}\mathbf{w})+\mathbf{s}(\mathbf{w},\boldsymbol{\nabla}\mathbf{w})=\mathbf{0}\ , \tag{7}$$

where

$$\mathbf{P}(\mathbf{w})=\frac{\partial\mathbf{q}(\mathbf{w})}{\partial\mathbf{w}}\in\mathbb{R}^m\otimes\mathbb{R}^m$$

is the Jacobian matrix of the transformation between the set of variables, and $\mathbf{q}=\mathbf{q}(\mathbf{w})$, $\mathbf{F}^{\mathrm{c}}(\cdot)$, $\mathbf{F}^{\mathrm{v}}(\cdot,\cdot)$, and $\mathbf{s}(\cdot,\cdot)$ in Eq. (6) are functionally different from those of Eq. (7) due to the different set of variables. An arbitrary smooth test function, $\mathbf{v}=\{v_1,\cdots,v_m\}$, multiplies Eq. (7), which is integrated by parts and written as

$$
\begin{aligned}
&\int_{\Omega} \mathbf{v} \cdot \left(\mathbf{P}(\mathbf{w}) \frac{\partial \mathbf{w}}{\partial t} \right) \, \mathrm{d}\Omega - \\
&\int_{\Omega} \nabla \mathbf{v} : \left(\mathbf{F}^{\mathrm{c}}(\mathbf{w}) + \mathbf{F}^{\mathrm{v}}(\mathbf{w}, \nabla \mathbf{w}) \right) \, \mathrm{d}\Omega + \\
&\int_{\partial\Omega} \mathbf{v} \otimes \boldsymbol{n} : \left(\mathbf{F}^{\mathrm{c}}(\mathbf{w}) + \mathbf{F}^{\mathrm{v}}(\mathbf{w}, \nabla \mathbf{w}) \right) \, \mathrm{d}\sigma + \\
&\int_{\Omega} \mathbf{v} \cdot \mathbf{s}(\mathbf{w}, \nabla \mathbf{w}) \, \mathrm{d}\Omega = \mathbf{0},
\end{aligned}
\tag{8}
$$

to obtain the weak formulation of the dG discretization of the governing equations, where $\boldsymbol{n}$ is the unit normal vector to the boundary. The domain is approximated with a computational mesh $\mathcal{T}_h = \{K\}$ of a set of non-overlapping and arbitrary shaped elements. A discontinuous finite element space with polynomial functions of degree at most l, here called solution approximation, continuous only inside each element K is defined on the mesh as

$$
\mathbf{V}_h^l \overset{\text{def}}{=} [\mathbb{P}_d^l(\mathcal{T}_h)]^m,
$$

$$
\mathbb{P}_d^l(\mathcal{T}_h) \overset{\text{def}}{=} \left\{ v_h \in L^2(\Omega_h) \, : \, v_h|_K \in \mathbb{P}_d^l(K), \forall K \in \mathcal{T}_h \right\}.
$$

In Eq. (8) the solution $\mathbf{w}$ and the test function $\mathbf{v}$ are replaced with a finite element approximation $\mathbf{w}_h$ and $\mathbf{v}_h$, respectively, of the space $\mathbf{V}_h^l$. Orthogonal and hierarchical basis functions are defined in the physical reference frame, as a satisfactory basis for the space $\mathbf{V}_h^l$, where the subscript d and the superscript m are here omitted in $\mathbf{V}_{h,d}^{m,l}$ for the sake of clarity. Any function $\mathbf{v}_h$ can be considered as a combination of the elements and of the basis. If $\varphi_i(\mathbf{x})$, with $i = 1, \dots, N_{\mathrm{DoF}}^K$ and $\mathbf{x} \in K$, are the basis functions of $\mathbf{V}_h^l$ for an arbitrary element K, and $w_{h,\alpha}(\mathbf{x}, t)$ and $v_{h,\alpha}(\mathbf{x})$ the αth component of the vector functions $\mathbf{w}_h(\mathbf{x}, t)$ and $\mathbf{v}_h(\mathbf{x})$, respectively, the dG spatial discretization of the governing equations can be written as

$$
\begin{aligned}
&\sum_{K \in \mathcal{T}_h} \int_K \varphi_i P_{\alpha\beta}(\mathbf{w}_h) \varphi_j \frac{\mathrm{d}w_{\beta j}}{\mathrm{d}t} \, \mathrm{d}\Omega + \\
&- \sum_{K \in \mathcal{T}_h} \int_K \frac{\partial \varphi_i}{\partial x_k} \left(F_{\alpha k}^{\mathrm{c}}(\mathbf{w}_h) + F_{\alpha k}^{\mathrm{v}}(\mathbf{w}_h, \boldsymbol{\nabla}_h \mathbf{w}_h + \mathbf{r}_h^g(\llbracket \mathbf{w}_h \rrbracket)) \right) \, \mathrm{d}\Omega + \\
&+ \sum_{f \in \mathcal{F}} \int_f \llbracket \varphi_i \rrbracket_k \left(\widehat{F}_{\alpha k}^{\mathrm{c}}(\mathbf{w}_h^{\pm}) + \widehat{F}_{\alpha k}^{\mathrm{v}} \left(\mathbf{w}_h^{\pm}, \left(\boldsymbol{\nabla}_h \mathbf{w}_h + \eta_f \mathbf{r}_h^f(\llbracket \mathbf{w}_h \rrbracket) \right)^{\pm} \right) \right) \, \mathrm{d}\sigma + \\
&+ \sum_{K \in \mathcal{T}_h} \int_K \varphi_i s_\alpha(\mathbf{w}_h, \boldsymbol{\nabla}_h \mathbf{w}_h + \mathbf{r}_h^g(\llbracket \mathbf{w}_h \rrbracket)) \, \mathrm{d}\Omega + \\
&+ \sum_{K \in \mathcal{T}_h} \int_K \varepsilon_p(\mathbf{w}_h^{\pm}, \mathbf{w}_h) \left(\frac{\partial \phi_h}{\partial x_k} b_k \right) \left(\frac{\partial w_{h,j}}{\partial x_k} b_k \right) \, \mathrm{d}\Omega = 0,
\end{aligned}
\tag{9}
$$

where repeated indices indicate an implicit sum over the repeated index range, namely $1, \dots, N_{\mathrm{DoF}}^K$ for indices i and j, $1, \dots, d$ for index k, and $1, \dots, m$ for indices α and β. $\mathbf{r}_h^g$ and $\mathbf{r}_h^f : [L^1(f)]^{m \times d} \to [\mathbb{P}_d^l(\mathcal{T}_h)]^{m \times d}$ are the global and local lifting operators used to guarantee a consistent, stable and accurate discretization of the viscous flux functions. f and $\mathcal{F}$ are the boundary or internal face of $\mathcal{T}_h$ and the sum of all the boundaries or internal faces of $\mathcal{T}_h$, respectively. The superscript $\pm$ is the trace of the solution of the two close elements sharing the face f. $\llbracket \cdot \rrbracket$ is the jump trace operator that is the vector for the scalar z, $\llbracket z \rrbracket = z^- \boldsymbol{n}^- + z^+ \boldsymbol{n}^+$. These definitions can be extended also to boundary faces for the weak imposition of boundary conditions.

The last term of Eq. (9) is the shock-capturing term, which is added to dissipate oscillations of the solution at high approximations, and introduces an artificial diffusion contribution within each element without the use of any shock sensor to find the oscillations. In particular, this term is always and everywhere active, but introduces numerical viscosity only in the elements with nonphysical oscillations. The artificial viscosity coefficient, ε_p, and the unit vector, $\mathbf{b}$, are written as

$$
\varepsilon_p(\mathbf{w}_h^{\pm}, \mathbf{w}_h) = C h_K^2 \frac{\left| s_p(\mathbf{w}_h^{\pm}, \mathbf{w}_h) \right| + \left| d_p(\mathbf{w}_h) \right|}{e^{\widetilde{p}}} f_p(\mathbf{w}_h), \quad \mathbf{b}(\mathbf{w}_h) = \frac{\nabla_h \widetilde{p}}{|\nabla_h \widetilde{p}| + \varepsilon},
$$

where

$$
s_p(\mathbf{w}_h^{\pm}, \mathbf{w}_h) = e^{\widetilde{p}} \frac{\partial \widetilde{p}}{\partial w_{h,\alpha}} s_\alpha(\mathbf{w}_h^{\pm}), \quad d_p(\mathbf{w}_h) = e^{\tilde{p}} \frac{\partial \widetilde{p}}{\partial w_{h,\alpha}} \left[\frac{\partial}{\partial x_k} \left(F_{\alpha k}^c(\mathbf{w}_h) \right) \right],
$$

$$f_p\left(\mathbf{w}_h\right) = \left|\nabla_h \tilde{p}\right| \left(\frac{h_K}{l}\right), \quad h_K = \frac{1}{\sqrt{\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2} + \frac{1}{(\Delta z)^2}}}.$$

ε is a small value of the machine precision order, h_K a geometrical factor, and $C = 0.2$. The term s_α is computed as the solution of the following problem:

$$\int_{\Omega_h} \phi_i s_\alpha \left(\mathbf{w}_h^{\pm}\right) d\mathbf{x} = \int_f \left[\phi_i\right]_k \left(\widehat{F}^c_{\alpha k}\left(\mathbf{w}_h^{\pm}\right) - F^c_{\alpha k}\left(\mathbf{w}_h\right)\right)^{\pm} d\sigma \quad i = 1, \ldots, N^K_{\mathrm{DoF}}.$$

The artificial viscosity coefficient, ε_p, controls the amount of numerical viscosity, and depends on the magnitude of the jump between the physical and numerical inviscid fluxes, s_p, and on the magnitude of the divergence of the inviscid flux vector inside elements, d_p.

The inviscid and viscous "numerical" flux functions $\widehat{F}^{\mathrm{c}}$ and $\widehat{F}^{\mathrm{v}}$ are introduced to uniquely define the flux on internal faces of $\mathcal{T}_h$, where the solution is discontinuous, and to guarantee the conservation of the numerical approximation, or to weakly set the boundary conditions on the boundary faces of $\mathcal{T}_h$. The inviscid flux is calculated from the exact solution of local Riemann problems, while the viscous flux is discretized according to the BR2 scheme proposed by Bassi and Rebay [30].

The system of Ordinary Differential Equations (ODEs) arising from the dG spatial discretization of the governing equations can be written in compact form as

$$\mathbf{M}\frac{\mathrm{d}\mathbf{W}}{\mathrm{d}t} + \mathbf{R}(\mathbf{W}) = \mathbf{0} \ , \tag{10}$$

where $\mathbf{R}$ is the residuals vector and $\mathbf{M}$ is the global block diagonal matrix. This matrix arises from the first integral of Eq. (9), and couples the DoFs of the different variables within the element through the transformation matrix, $\mathbf{P}$. Here the focus is on steady-state simulations, where $\mathbf{W}$ is the solution of the nonlinear algebraic problem $\mathbf{R}(\mathbf{W}) = \mathbf{0}$. In practice, the solution $\mathbf{W}$ can be found with the integration in time of Eq. (10), starting from an initial solution until the convergence under a tolerance set by the user.

Equation (10) can be integrated in time by means of the Linearized Backward Euler (LBE) scheme as

$$\left[\frac{\mathbf{M}^n}{\Delta t} + \frac{\partial \mathbf{R}(\mathbf{W}^n)}{\partial \mathbf{W}}\right] \Delta \mathbf{W}^n = -\mathbf{R}(\mathbf{W}^n) \ , \tag{11}$$

where the linear system for each time-step is solved using the matrix-explicit GMRES algorithm. To improve the convergence of GMRES, different preconditioners can be used, e.g., the Block Jacobi (BJ) with one block per process and ILU(0) factorization, and the Additive Schwartz Method (ASM). However, the influence of the preconditioner on the performance of the FMG algorithms in the simulations here carried out is negligible. The linear algebra and parallelization are based on the PETSc [31] library and MPI. The different user-defined parameters of the GMRES algorithm are: the relative/absolute tolerance, tol_{GMRES}, the Krylov spaces, n_{kry}, and the maximum number of iterations, $n_{iter,max}$. $tol_{GMRES} = 10^{-12}$ is always used as relative tolerance for the SO time integration and the finest level of the p-MG algorithms. At lower levels the tolerance is automatically increased with respect to the value of the finest level, $l^{\max}$, as $10^{(l^{\max} - l^{\mathrm{l}})} tol_{GMRES}$, where l^{l} is the current level. $n_{kry} = 240$ and $n_{iter,max} = 480$ are used for the SO time integration and the finest level of the p-MG algorithms. At lower levels the Krylov spaces and the maximum number of iterations are reduced with respect to the finest level value as $n_{kry}/(l^{max} - l^l)$ and $n_{iter,max}/(l^{max} - l^l)$.

In order to advance the solution to the steady-state, a pseudo-transient continuation strategy is used, where the local pseudo-time-step is calculated as

$$\Delta t_K = \mathrm{CFL}\frac{h_{K,\mathrm{CFL}}}{c_v + d_v}, \qquad c_v = |\mathbf{v}| + a, \qquad d_v = 2\frac{\mu_e + \lambda_e}{h_{K,\mathrm{CFL}}}, \qquad h_{K,\mathrm{CFL}} = d\frac{\Omega_K}{S_K},$$

where c_v, d_v are the inviscid and diffusive velocities, $h_{K,\mathrm{CFL}}$ is the reference length of the element K, μ_e and λ_e are the effective dynamic viscosity and conductivity, while Ω_K and S_K are the volume and the surface of the element K. The algorithm to increase the CFL number as the residual decreases is far from trivial, especially for RANS and turbulent model equations. The CFL law can be written as

$$\mathrm{CFL} = \begin{cases} \mathrm{CFL}_{\min}/\xi^{\alpha} & \text{if} \quad \xi \le 1 \\ \mathrm{CFL}_{\mathrm{ord}} + \Phi(\xi)(\mathrm{CFL}_{\min} - \mathrm{CFL}_{\mathrm{ord}}) & \text{if} \quad \xi > 1 \end{cases} \tag{12}$$

with

$$\xi = \begin{cases} \min\left(1, \xi_2\right) & \text{if} \quad \xi_\infty \le 1, \qquad \xi_2 \overset{\mathrm{def}}{=} \max\limits_{\alpha=1,\ldots,m} \left(||R_\alpha||_2 / ||R_{\alpha 0}||_2\right), \\ \xi_\infty & \text{if} \quad \xi_\infty > 1, \qquad \xi_\infty \overset{\mathrm{def}}{=} \max\limits_{\alpha=1,\ldots,m} \left(||R_\alpha||_\infty / ||R_{\alpha 0}||_\infty\right), \end{cases}$$

where $||\cdot||_2$ and $||\cdot||_\infty$ are the L_2 and L_∞ norms, R_α is the residual vector of the αth equation, $R_{\alpha 0}$ is the corresponding residual at the first iteration, and

$$\Phi(\xi) = \exp\left(\frac{\alpha(1-\xi)(l^{\max} - l^{\mathrm{l}} + 1) n_{\mathrm{sf}} \mathrm{CFL}_{\min}}{\mathrm{CFL}_{\min} - \mathrm{CFL}_{\mathrm{ord}}}\right) \ .$$

In the CFL law of Eq. (12) there are different user-defined parameters: $\mathrm{CFL}_{\mathrm{min}}$, $\mathrm{CFL}_{\mathrm{ord}}$, and α. $\mathrm{CFL}_{\mathrm{min}}$ is the minimum value of the CFL number, $\mathrm{CFL}_{\mathrm{ord}} = 1/(2l+1)$ is the maximum CFL number for explicit multistage schemes, and α sets the growth rate of the CFL number. $\Phi(\xi)$ is based on the term $(l^{\max} - l^{\mathrm{l}} + 1)n_{\mathrm{sf}}$, which introduces the dependence on the level l of the p-MG algorithm in the CFL law. In particular, at lower levels the CFL number is automatically increased with respect to the value of the finest level, and the increment is proportional to the difference between the finest level, $l^{\max}$, and the current level l^{l}. Finally, n_{sf} is a safety factor to adjust $\Phi(\xi)$ between levels and, eventually, to increase even more the CFL number at the lowest level. For the implicit (single-order) time integration $l^{\max} = l^{\mathrm{l}}$.

4. *p*-MG algorithm

The performance of single-order time integration schemes deteriorates after a few iterations, and this can be ascribed to the frequency content of the error in the solution. In fact, first iterations allow the damping of errors whose wavelengths are comparable to the dimensions of the elements of the mesh, while leaving unchanged smooth and longer wavelength errors that slow down the convergence. The *h*- and *p*-MG algorithms couple the reduction of high-frequency error components with single-order time integration schemes, e.g., the relaxation schemes or smoothers, and of low-frequency error components by a coarse correction term. These algorithms proved to be effective for the improvement of the convergence to the steady-state solution for both linear and nonlinear problems.

In the algorithm, the smoother, or relaxation scheme, at an arbitrary level, must damp all the error modes that cannot be represented on "coarser" levels, where "coarse" is a lower solution approximation in the *p*-MG algorithms. Different paths can be used to move between the different levels during the convergence process (the most popular in literature are V- and W-cycle). The algorithm is based on a recursive application of the two-level algorithm, where the exact solution on the coarser level is used to accelerate the solution on the finer level. In practice, the expansive exact solution on the coarse level is never calculated, and the two-level algorithm is recursively applied to progressively coarser levels.

At each level, a number of pre-/post-smoothing iterations (ν_{pre}/ν_{post}) allow restricting/prolongating the solution to the next coarser/finer level. As a further improvement, in the FMG algorithm the coarser level solutions are exploited to obtain a good initial guess to initialize the computation of the finer grids. In general, it is not necessary to fully converge on each level before switching to the next finer level, since the corresponding discretization error can be relatively large. A residual-based criterion [6] is used to switch to the next finer level, i.e., the solution $\mathbf{W}^l$ is prolongated to the level $l+1$ when $\mathrm{TOL} = \max(||c^l||_2) < 10^{-7}$. $||\mathbf{R}^l||_2$ is the L^2 norm of residual components, and TOL has the same value adopted for implicit (single-order) simulations to switch from low- to high-order solutions, when low-order simulations are used as the initial solution for high-order simulations.

4.1. FAS FMG V-cycle algorithm

Algorithm 1 shows the FMG algorithm based on the FAS V-cycle between $l = 1$ and l^{max}. Furthermore, Fig. 1 shows a schematic representation of both FMG and FAS V-cycle algorithms, where $l^{max} = 4$. The steady form of the discretized governing equations, see Eq. (10), for a generic level l can be written as

$$\mathbf{R}^l\left(\mathbf{W}^l\right) = \mathbf{s}^l, \tag{13}$$

where $\mathbf{W}^l$ is the solution vector and $\mathbf{s}^l$ is the source term. $\mathbf{R}^l$ is discretized on each level according to the governing equations, while the source term $\mathbf{s}^l$ is projected from the finer level ($\mathbf{s}^l = 0$ on the finest level). The discrete residual can be defined as

$$\mathbf{r}^l = \mathbf{s}^l - \mathbf{R}^l\left(\mathbf{W}_n^l\right),$$

where $\mathbf{W}_n^l$ is the vector solution $\mathbf{W}^l$ at the iteration n. In the basic two-level algorithm between a coarse and fine level, the exact solution on the coarse level $l-1$ is used to correct the solution on the fine level l. The solution $\mathbf{W}_n^l$ and the discrete residual $\mathbf{r}^l$ are restricted to the coarse level as

$$\mathbf{W}_{n,0}^{l-1} = \widetilde{\mathbf{I}}_l^{l-1}\mathbf{W}_n^l, \qquad \mathbf{r}^{l-1} = \mathbf{I}_l^{l-1}\mathbf{r}^l\left(\mathbf{W}_n^l\right),$$

where $\widetilde{\mathbf{I}}_l^{l-1}$ and $\mathbf{I}_l^{l-1}$ are the restriction operators for the solution and the discrete residual, respectively. On the coarse level the source, or forcing, term is calculated as

$$\mathbf{s}^{l-1} = \mathbf{r}^{l-1} + \mathbf{R}^{l-1}\left(\mathbf{W}_0^{l-1}\right),$$

where $\mathbf{W}_{n,0}^{l-1}$ is the solution restricted to the coarse level before the smoothing iteration. A number of post-smoothing iterations ν_{post} is carried out on the coarse level to compute the solution $\mathbf{W}_n^{l-1}$ and the correction, or error, term as

$$\mathbf{e}^{l-1} = \mathbf{W}_n^{l-1} - \mathbf{W}_{n,0}^{l-1}.$$

The correction term is prolongated to the fine level, i.e., $\mathbf{e}^l = \widetilde{\mathbf{I}}_{l-1}^l\mathbf{e}^{l-1}$, and used to correct the solution and improve the convergence on the fine level as

$$\mathbf{W}_{n+1}^l = \mathbf{W}_n^l + \widetilde{\mathbf{I}}_{l-1}^l\mathbf{e}^{l-1},$$

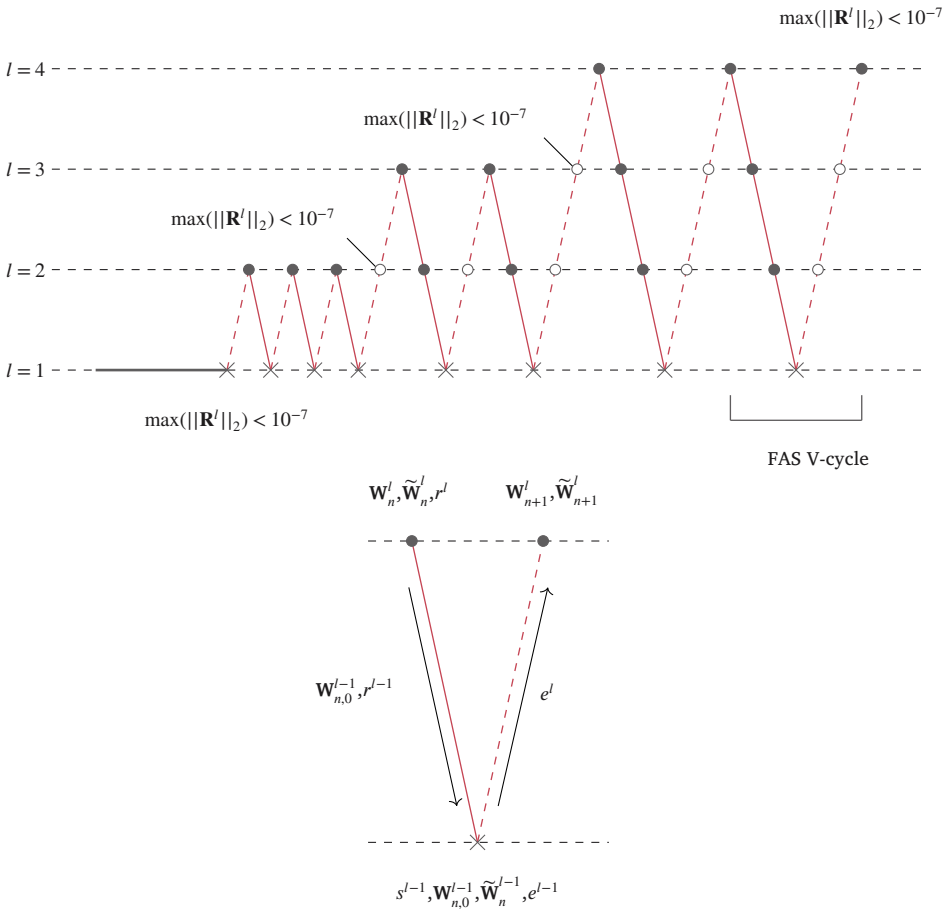


Fig. 1. FMG algorithm based on V-cycle between levels $l = 1$ and $l = 4$, e.g., between $\mathbb{P}^0$ and $\mathbb{P}^3$. $\mathbf{W}_n^l$ is the solution before smoothing, $\widetilde{\mathbf{W}}_n^l$ the solution after smoothing, $\mathbf{W}_{n,0}^{l-1}$ the solution restricted on the lower level, r^l the discrete residual, s^l the source (or forcing) term, and e^l the correction (or error) term. ● ν_{pre} pre-smoothing iterations, ○ ν_{post} post-smoothing iterations, × ν_c iterations, - - - prolongation of the correction term to the next finer level, —— restriction of the solution and the discrete residual to the next coarser level, —— implicit (single-order) time integration.

where $\widetilde{\mathbf{I}}_{l-1}^l$ is the prolongation operator for the solution. The time dependent term, $\mathbf{M}/\Delta t$, is only used to avoid instabilities and increase the robustness in the first part of steady-state simulations, and Eq. (13)) and can be rewritten as

$$\left[\frac{\mathbf{M}^l}{\Delta t^l} + \frac{\partial \mathbf{R}^l\left(\mathbf{W}_n^l\right)}{\partial \mathbf{W}_n^l}\right]\left(\mathbf{W}_{n+1}^l - \mathbf{W}_n^l\right) = -\mathbf{R}^l\left(\mathbf{W}_n^l\right) + \mathbf{s}^l. \tag{14}$$

4.2. Efficient and robust implementation for the RANS equations

The computational efficiency and robustness of h- and p-MG algorithms can decrease drastically when RANS equations coupled with a turbulence model are solved. This reduction is associated with the stiffness introduced by the RANS equations, and, in particular, by the turbulence model, and by highly stretched meshes used for an accurate resolution of turbulent boundary layers. Furthermore, this reduction can also be explained by the inability to discretize the source terms of the turbulence model on the coarser levels of the algorithm.

To alleviate these problems a special treatment for the turbulence model source terms is proposed. The residual $\mathbf{R}^l$ for a generic level l can be split into the terms associated with the inviscid and diffusive fluxes, $\mathbf{R}_f$, and to the turbulence model, $\mathbf{R}_t$, i.e.,

$$\mathbf{R}^l\left(\mathbf{W}^l\right) = \mathbf{R}_f^l\left(\mathbf{W}^l\right) + \mathbf{R}_t^l\left(\mathbf{W}^l\right), \tag{15}$$

and on the coarse level of the basic two-level algorithm can be written as

$$\mathbf{R}^{l-1}\left(\mathbf{W}^{l-1}\right) \approx \mathbf{R}_f^{l-1}\left(\mathbf{W}^{l-1}\right) + \mathbf{R}_t^{l-1}\left(\mathbf{W}_0^{l-1}\right), \tag{16}$$

Algorithm 1 FAS V-cycle (top) and FMG (bottom) algorithm between $l = 1$ and $l = l^{max}$, where the difference between the solution before and after the smoothing iterations, $\mathbf{W}_n^l$ and $\widetilde{\mathbf{W}}_n^l$, and the subscript n for the number of iteration are omitted for the sake of clarity.

procedure FAS-V($l, \mathbf{W}^l, \mathbf{r}^l, l_{max}$)
  $\mathbf{W}_0^l \leftarrow \mathbf{W}^l$
  if $l < l_{max}$ **then**
    $\mathbf{s}^l = \mathbf{r}^l + \mathbf{R}^l(\mathbf{W}_0^l)$
  end if
  $W^l \leftarrow SMOOTH(\mathbf{W}^l, \nu_{pre})$
  if $l > 1$ **then**
    $\mathbf{W}^{l-1} \leftarrow \widetilde{\mathbf{I}}_l^{l-1}\mathbf{W}^l$
    $\mathbf{r}^{l-1} \leftarrow \mathbf{I}_l^{l-1}\mathbf{r}^l$
    call FAS-V($l-1, \mathbf{W}^{l-1}, \mathbf{r}^{l-1}, l_{max}$)
    $\mathbf{W}^l = \mathbf{W}^l + \widetilde{\mathbf{I}}_{l-1}^l\mathbf{e}^{l-1}$
  end if
  $\mathbf{W}^l \leftarrow SMOOTH(\mathbf{W}^l, \nu_{post})$
  if $l < l_{max}$ **then**
    $\mathbf{e}^l = \mathbf{W}^l - \mathbf{W}_0^l$
  end if
end procedure

procedure FAS-FMGV($l, \mathbf{W}^l$,TOL)
  if $l > 1$ **then**
    $\mathbf{W}^{l-1} \leftarrow \widetilde{\mathbf{I}}_l^{l-1}\mathbf{W}^l$
    call FAS-FMGV($l-1, \mathbf{W}^{l-1}$,TOL)
    $\mathbf{W}^l = \widetilde{\mathbf{I}}_{l-1}^l\mathbf{W}^{l-1}$
    while $\|\mathbf{r}\| > TOL$ **do**
      call FAS-V($l, \mathbf{W}^l, \mathbf{r}^l, l$)
      $\mathbf{r}^l \leftarrow \mathbf{s}^l - \mathbf{R}^l(\mathbf{W}^l)$
    end while
  else
    while $\|\mathbf{r}\| > TOL$ **do**
      $W^l \leftarrow SMOOTH(\mathbf{W}^l, \nu_c)$
      $\mathbf{r}^l \leftarrow \mathbf{s}^l - \mathbf{R}^l(\mathbf{W}^l)$
    end while
  end if
end procedure

where the turbulence term, $\mathbf{R}_t^{l-1}\left(\mathbf{W}_0^{l-1}\right)$, is calculated with the restricted solution $\mathbf{W}_0^{l-1}$, and is frozen during the iterations on the coarse level. Equation (14) can be rewritten for the level $l-1$ as

$$\left[\frac{\mathbf{M}^{l-1}}{\Delta t^{l-1}} + \frac{\partial \mathbf{R}^{l-1}\left(\mathbf{W}_k^{l-1}\right)}{\partial \mathbf{W}_k^{l-1}}\right]\left(\mathbf{W}_{k+1}^{l-1} - \mathbf{W}_k^{l-1}\right) = -\mathbf{R}_f^{l-1}\left(\mathbf{W}_k^{l-1}\right) - \mathbf{R}_t^{l-1}\left(\mathbf{W}_0^{l-1}\right) + \mathbf{r}^{l-1} + \mathbf{R}^{l-1}\left(\mathbf{W}_0^{l-1}\right), \tag{17}$$

and

$$\left[\frac{\mathbf{M}^{l-1}}{\Delta t^{l-1}} + \frac{\partial \mathbf{R}^{l-1}\left(\mathbf{W}_k^{l-1}\right)}{\partial \mathbf{W}_k^{l-1}}\right]\left(\mathbf{W}_{k+1}^{l-1} - \mathbf{W}_k^{l-1}\right) = -\mathbf{R}_f^{l-1}\left(\mathbf{W}_k^{l-1}\right) \cancel{-\mathbf{R}_t^{l-1}\left(\mathbf{W}_0^{l-1}\right)} + \mathbf{r}^{l-1} + \mathbf{R}_f^{l-1}\left(\mathbf{W}_0^{l-1}\right) + \cancel{\mathbf{R}_t^{l-1}\left(\mathbf{W}_0^{l-1}\right)}, \tag{18}$$

where this choice avoids the computation of the turbulence model source term on the coarser level. In fact, the turbulence source term is computed only on the finest level and is restricted to the coarser levels through the term $\mathbf{r}^{l-1}$. The treatment for the turbulent part of the residual does not affect the memory consumption, as no additional arrays are created. Two different routines are used for the assembly of the residual. On the finest level, the complete residual (the convective and diffusive part, and the turbulent source term) is computed, while on coarser levels the assembly of the contribution associated to the source term of the turbulence model is skipped.

Furthermore, Eqs. (14) and (18) show that on each level the Jacobian matrix $\mathbf{J} = \partial \mathbf{R}(\mathbf{W})/\partial \mathbf{W}$ or some subset, e.g., the block diagonal part, must be calculated, and this drastically influences the performance of the algorithm. To alleviate this problem, a special treatment of the Jacobian matrix is proposed, and, in particular, the Jacobian matrix is calculated only one time on the finest level and is restricted to the coarser levels as

$$\mathbf{J}^{l-1} = \widehat{\mathbf{I}}_l^{l-1}\mathbf{J}^l,$$

where $\widehat{\mathbf{I}}_l^{l-1}$ is the matrix restriction operator. This choice does not have a negative influence on the robustness, but guarantees a huge reduction of the computational cost of the simulations.

4.3. Restriction and prolongation operators

The restriction and prolongation operators for the solution and the correction term are the L^2 projections in the spaces $\mathbf{V}_h^{l-1}$ and $\mathbf{V}_h^l$, respectively. The low-order solution is calculated from the high-order solution with the condition

$$\int_\Omega \mathbf{v}_h^{l-1}\mathbf{W}_h^{l-1}d\Omega = \int_\Omega \mathbf{v}_h^{l-1}\mathbf{W}_h^{l}d\Omega, \quad \forall \mathbf{v}_h^{l-1} \in \mathbf{V}_h^{l-1}, \tag{19}$$

which can be rewritten as

$$\mathbf{M}^{l-1}\mathbf{W}^{l-1} = \mathbf{M}_l^{l-1}\mathbf{W}^{l-1}, \tag{20}$$

where

$$\mathbf{M}^{l-1} = \left[M_{ij}\right]^{l-1} = \int_\Omega \phi_i^{l-1}\phi_j^{l-1}d\Omega, \qquad \mathbf{M}_l^{l-1} = \left[M_{ij}\right]_l^{l-1} = \int_\Omega \phi_i^{l-1}\phi_j^{l}d\Omega.$$

Thanks to Eq. (20), $\mathbf{W}^{l-1}$ can be calculated from $\mathbf{W}^l$ as

$$\mathbf{W}^{l-1} = \widetilde{\mathbf{I}}_l^{l-1}\mathbf{W}^l, \qquad \widetilde{\mathbf{I}}_l^{l-1} \equiv (\mathbf{M}^{l-1})^{-1}\mathbf{M}_l^{l-1}, \tag{21}$$

which represents the solution restriction operator $\widetilde{\mathbf{I}}_l^{l-1}$. Similarly, the high-order correction term is calculated from the low-order correction term with the condition

$$\mathbf{M}^{l}\mathbf{e}^{l} = \mathbf{M}_{l-1}^{l}\mathbf{e}^{l-1}, \tag{22}$$

and $\mathbf{e}^l$ can be calculated from $\mathbf{e}^{l-1}$ as

$$\mathbf{e}^{l} = \widetilde{\mathbf{I}}_{l-1}^{l}\mathbf{e}^{l-1}, \qquad \widetilde{\mathbf{I}}_{l-1}^{l} \equiv (\mathbf{M}^{l})^{-1}\mathbf{M}_{l-1}^{l} = (\mathbf{M}^{l})^{-1}(\mathbf{M}_l^{l-1})^{T}. \tag{23}$$

The choice of orthogonal and hierarchical modal expansion bases for the discretization allows to write the restriction and prolongation operators $\widetilde{\mathbf{I}}_l^{l-1}$ and $\widetilde{\mathbf{I}}_{l-1}^{l}$ as $\widetilde{\mathbf{I}}_l^{l-1} = \delta_{l,l-1}$ and $\widetilde{\mathbf{I}}_{l-1}^{l} = \delta_{l-1,l}$, respectively, where δ_{ij} is the Kronecker symbol. The DoFs of the restricted solution are the same as the low-order subset of the high-order solution, while the DoFs of the prolongated correction term are the same as the low-order correction term with null high-order components.

The residuals at level l can be rewritten as $\mathbf{R}_j^l = R(\phi_j^l, \mathbf{W}^l)$ to show its dependence on the shape function and the solution. The restricted residual $\mathbf{R}_j^{l-1}$ can be written as a linear combination of weighted residuals at level l, i.e.,

$$\mathbf{R}_i^{l-1} = R(\phi_i^{l-1}, \mathbf{W}^l) = R(\alpha_{ij}\phi_j^l, \mathbf{W}^l) = \alpha_{ij}R(\phi_j^l, \mathbf{W}^l) = \alpha_{ij}\mathbf{R}_j^l,$$

or

$$\mathbf{R}^{l-1} = \boldsymbol{\alpha}\mathbf{R}^l = \left(\widetilde{\mathbf{I}}_{l-1}^{l}\right)^T\mathbf{R}^l,$$

which shows that $\mathbf{I}_l^{l-1} = (\widetilde{\mathbf{I}}_{l-1}^{l})^T$. For this reason, the choice of orthogonal and hierarchical modal expansion bases for the discretization allows also to write the operators $\mathbf{I}_l^{l-1}$ as

$$\mathbf{I}_l^{l-1} = \delta_{l,l-1}, \tag{24}$$

i.e., the DoFs of the restricted residual are the same of the low-order subset of the high-order residual. Finally, the Galerkin coarse grid operator is chosen as matrix restriction operator, and is defined as

$$\widehat{\mathbf{I}}_l^{l-1}[\quad] = \widetilde{\mathbf{I}}_l^{l-1}[\quad]\widetilde{\mathbf{I}}_{l-1}^{l}, \tag{25}$$

where [ ] represents the matrix to restrict.

4.4. Automatic choice of the smoothing iterations and "skip condition"

The number of the pre- and post-smoothing iterations, ν_{pre} and ν_{post}, is crucial for the performance of h- and p-MG algorithms, for both robustness and computational efficiency. The smoothing iterations should be enough to damp the high-frequency modes of the solution error before the restriction of the solution to the next coarser level, and the prolongation to the next finer level, respectively. On the other hand, a higher number of smoothing iterations increases the computational cost of the simulations. In literature, almost all authors set the number of the pre- and post-smoothing iterations, which are the same for each level [32,10]. This choice does not guarantee an optimization of the performance and needs a "trial and error" approach to find the best values.

Recently, Rueda-Ramírez et al. [22] proposed a residual-based criterion to automatically set the number of smoothing iterations. The authors coupled the p-MG and p-adaptation algorithms, where the p-MG algorithm is used both as a solver and as a truncation error estimator. The solver, based on a Discontinuous Galerkin Spectral Element Method (DGSEM), is validated for the NS equations

and, in particular, for the two-dimensional laminar boundary layer on a flat plate and the three-dimensional flow around a sphere. Here, a slightly modified version of the residual-based criterion of Rueda-Ramírez et al. [22] for the number of the pre- and post-smoothing iterations, ν_{pre} and ν_{post}, is used to reduce the number of iterations and to avoid the manual choice of the optimal values. In fact, this criterion compares the norm of the coarse level residual $\|\mathbf{R}^{l-1}\|_\infty$ during pre- and post-smoothing iterations with the restricted residual $\|\mathbf{I}_l^{l-1}\mathbf{r}^l\|_\infty$, while Rueda-Ramírez et al. [22], during post-smoothing, compare the residual with the last residual of the pre-smoothing. During pre-smoothing, the criterion is

$$\|\mathbf{R}^{l-1}\|_\infty < \eta\|\mathbf{I}_l^{l-1}\mathbf{r}^l\|_\infty, \tag{26}$$

while during the post-smoothing is

$$\|\mathbf{R}^{l-1}\|_\infty \leq \|\mathbf{I}_l^{l-1}\mathbf{r}^l\|_\infty, \tag{27}$$

where $\eta = 1.1$. A maximum number of pre-, $\nu_{pre,max}$ and post-smoothing, $\nu_{post,max}$, iterations is set. On the coarsest level, a constant value, $\nu_c = 5$, is always used for all simulations, as its value shows a negligible influence on the computational cost. If Eq. (26) or (27) is not satisfied, the MG correction is kipped, i.e., the V-cycle ends early and the computation jumps to the post smoothings on the finest level.

5. Smoothers

To increase the performance of the p-MG algorithm for stiff problems, such as with the RANS and turbulence model equations, semi-implicit smoothers, e.g., element-block Jacobi/Gauss–Seidel, can be used instead of the traditional Runge-Kutta (RK) schemes [6]. Moreover, additional stiffness associated with highly stretched meshes used for an accurate resolution of turbulent boundary layers can be alleviated with directional refinement or line smoothers. In the h-MG algorithm, the mesh is coarsened only in the direction normal to the mesh stretching, or alternatively, a line smoother is used in the direction normal to the mesh stretching. In the p-MG framework, the equivalent of the directional refinement, i.e., the decrease of the solution approximation only in the direction normal to the mesh stretching, is difficult to implement, while many authors [21,6] demonstrated the effectiveness of line smoothers.

In this work, different smoothers are considered: (i) the Block Implicit (BIRK) scheme, see Algorithm 2, (ii) the Line Implicit Runge-Kutta (LIRK) scheme, see Algorithm 3, and (iii) the Linearized Backward Euler (LBE) scheme, see Algorithm 5. Implicit smoothers, e.g., LBE, are the best choice when the computational cost is the main goal, even if they have a higher memory footprint in comparison to BIRK and LIRK. In particular, two different smoothing strategies are compared: (i) LBE on all levels (here called LBE), and (ii) LIRK on the finest level to reduce the influence of the anisotropic mesh in the boundary layer, BIRK on the intermediate levels to reduce the memory footprint, and LBE on the coarsest level (here called BIRK/LIRK).

Algorithm 2 The Block-Implicit Runge-Kutta (BIRK) smoother.

$\mathbf{W}_0^l = \mathbf{W}_n^l$
for $k = 1, m$ **do**
  Solve $\left[\dfrac{\mathbf{M}^l}{\Delta t^l} + \alpha_m \mathbf{D}^l\left(\mathbf{W}_0^l\right)\right]\delta\mathbf{W}_k = -\alpha_k \mathbf{R}^l\left(\mathbf{W}_{k-1}^l\right)$
  $\mathbf{W}_k^l = \mathbf{W}_{k-1}^l + \delta\mathbf{W}_k$
end for
return $\mathbf{W}_{n+1}^l = \mathbf{W}_m^l$

Algorithm 3 The Line-Implicit Runge-Kutta (LIRK) smoother.

$\mathbf{W}_0^l = \mathbf{W}_n^l$
for $k = 1, m$ **do**
  Solve $\left[\dfrac{\mathbf{M}^l}{\Delta t^l} + \alpha_m \mathbf{T}^l\left(\mathbf{W}_0^l\right)\right]\delta\mathbf{W}_k^l = -\alpha_k \mathbf{R}^l\left(\mathbf{W}_{k-1}^l\right)$
  $\mathbf{W}_k^l = \mathbf{W}_{k-1}^l + \delta\mathbf{W}_k$
end for
return $\mathbf{W}_{n+1}^l = \mathbf{W}_m^l$

In the BIRK smoother, the matrix of the linear system, $\mathbf{M}^l/\Delta t^l + \alpha_m\mathbf{D}^l$, is calculated only for the first Runge-Kutta stage, and $\mathbf{D}^l(\mathbf{W}^l)$ is the block diagonal part of the Jacobian matrix, $\mathbf{J}^l$. In the LIRK smoother, the matrix of the linear system, $\mathbf{M}^l/\Delta t^l + \alpha_m\mathbf{T}^l$, is calculated only for the first Runge-Kutta stage, and $\mathbf{T}^l(\mathbf{W}^l)$ is the block tridiagonal matrix associated with a line. The lines are generated with Algorithm 4 only in flow regions with stretched elements. For each line, a block tridiagonal system is assembled and solved efficiently with the Thomas algorithm [33]. For parallel computations, the use of a line smoother needs an ad-hoc domain partitioning algorithm to avoid the cut of the lines, and the loss of performance.

Algorithm 4 Line creation algorithm for the LIRK smoother, nb is the number of boundary faces, $\boldsymbol{n}_f$ the normal to face f, e_1 the element adjacent to the boundary face, ne_{max} the maximum number of elements per line, e_i the element close to the element e_{i-1} on the line, $\mathrm{AR}_{e_i} = \min(S_{e_i,j})/\max(S_{e_i,j})$ the aspect ratio of an element e_i, where $S_{e_i,j}$ is the surface of the j-th face of the element e_i.

```
for f = 1, nb do
    Set l_f(ne_max)
    Compute n_{l_f}, e_1
    l_f(1) = e_1
    for i = 2, ne_max do
        if e_i ∩ n_f then
            l_f(i) = e_i
            Compute AR_{e_i} and AR_{e_{i-1}}
            if AR_{e_i}/AR_{e_{i-1}} < 0.1 or i = ne_max then
                Exit
            end if
        end if
    end for
end for
```

Algorithm 5 The linearized Backward Euler (LBE) smoother.

Solve $\left[\dfrac{\mathbf{M}^l}{\Delta t^l} + \mathbf{J}^l\left(\mathbf{W}_n^l\right)\right]\delta\mathbf{W}_n = \mathbf{R}^l\left(\mathbf{W}_n^l\right)$

return $\mathbf{W}_{n+1}^l = \mathbf{W}_n^l + \delta\mathbf{W}_n$

6. Benchmark cases

The following test cases are used to assess the performance of the proposed p-MG algorithm with different smoothers and strategies to choose the smoothing iterations:

(i) the streamlined three-dimensional body (BTC0) from the ADIGMA European project, based on a 10% thick airfoil with boundaries generated by a surface of revolution, at Mach number $\mathrm{M} = 0.5$ and Reynolds number $\mathrm{Re}_L = 10^7$, based on the length of the body, L, and the free-stream flow conditions, and angle of attack $\alpha = 5°$,

(ii) the NASA 65° sweep DW from the 2*nd* International Vortex Flow Experiment (VFE-2) [34], at Mach number $\mathrm{M} = 0.4$ and Reynolds number $\mathrm{Re}_c = 3 \times 10^6$, based on the main aerodynamic chord, c, and the free-stream flow conditions, and angle of attack of $\alpha = 13.3°$.

The maximum solution approximation is $\mathbb{P}^6$. The FMG algorithm is used with different smoothers to assess their performance. Moreover, the influence of the following parameters is investigated: the number of smoothing iterations at the coarsest level, ν_c, the number of the pre- and post-smoothing iterations for intermediate and finest levels, ν_{pre} and ν_{post}, the tolerance of the GMRES algorithm, tol_{GMRES}, and the initialization of the solution. The performance of the standard FMG is compared with a version that automatically sets the number of pre- and post-smoothing iterations, here called α-FMG. The number of smoothing iterations is defined manually to find the best compromise for the FMG algorithm. All the tables show also the number of smoothing iterations for the α-FMG algorithm, which is averaged on the different levels and cycles.

Notice that only the LBE smoother allows the initialization of the solution with the free-stream flow conditions, i.e., $\mathbb{P}^{coarsest} = \mathbb{P}^0$, while other smoothers, e.g., BIRK and LIRK, do not allow to use $\mathbb{P}^{coarsest} = \mathbb{P}^0$, as proved by Ghidoni et al. [20,21,23] and Colombo et al. [6]. As a consequence, the comparison between the different smoothers can be done only with the $\mathbb{P}^{coarsest} = \mathbb{P}^1$.

A residual-based criterion [6] is used to switch to the next finer level, i.e., the solution $\mathbf{W}^l$ is prolongated to the level $l+1$ when $\mathrm{TOL} = \max(||\mathbf{R}^l||_2) < 10^{-7}$. $||\mathbf{R}^l||_2$ is L^2 norm of the residual components, and TOL has the same value adopted for implicit (single-order) simulations to switch from low- to high-order solutions, when low-order simulations are used as the initial solution for high-order simulations.

The computational cost, t_{CPU}, used in all the tables is a normalized value with respect to the Integrated Performance Analysis of Computer Systems (IPACS) value, $t_{TauBench}$ [35]. The cluster used for all simulations is a 2×CPU Intel CascadeLake 8260, with 24 cores each, 2.4 GHz, 384 GB RAM and is characterized by the benchmark value $t_{TauBench} = 10.32s$.

6.1. BTC0

The first test case is the turbulent flow around a BTC0 from the ADIGMA European project, based on a 10% thick airfoil with boundaries generated by a surface of revolution, at Mach number $\mathrm{M} = 0.5$, Reynolds number $\mathrm{Re}_L = 10^7$, and angle of attack $\alpha = 5°$. The Reynolds number is based on the length of the body, L, and the free-stream flow conditions, ν_∞ and u_∞.

The domain is a box of dimension $20L \times 20L \times 10L$, where x is the stream-wise direction, y the cross-wise direction, and z the span-wise direction. A no-slip condition is applied on the body; a symmetric half-body configuration is considered for the simulations, and a symmetry condition is prescribed on the x-y plane. Different meshes are considered to study the performance of the algorithm and smoothers with meshes characterized by a different number of elements and stretching of the elements, AR. The coarse and fine meshes have 832 and 6 656 quadratic hexahedral elements with maximum AR of 8 850 and 20 000, respectively (see Fig. 2).

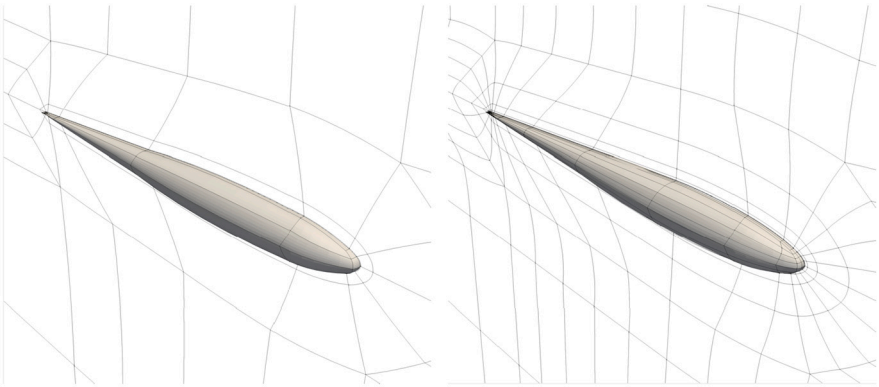


Fig. 2. BTC0. Detail of the coarse (left) and fine (right) mesh on the body and the symmetry plane, with 832 and 6656 quadratic hexahedral elements.

Table 1
BTC0. Performance of different smoothers on the coarse mesh, $tol_{GMRES} = 10^{-12}$, $CFL_{min} = 1$, $\mathbb{P}^{coarsest} = \mathbb{P}^1$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, and Δt_{CPU} the reduction due to the use of the FMG algorithms in comparison to the traditional implicit time integration.

$\mathbb{P}^l$			ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
	LBE	SO	-	-	152	1.88×10^1	-
	LBE	FMG	1	5	65	7.46×10^0	−60.30%
2	BIRK/LIRK3	FMG	10	10	55	6.02×10^0	−67.98%
	BIRK/LIRK4	FMG	5	5	57	5.80×10^0	−69.14%
	BIRK/LIRK5	FMG	1	10	56	6.74×10^0	−64.16%
	LBE	SO	-	-	216	1.28×10^2	-
	LBE	FMG	2	2	68	3.65×10^1	−71.50%
3	BIRK/LIRK3	FMG	1	5	58	3.04×10^1	−76.22%
	BIRK/LIRK4	FMG	5	5	58	3.35×10^1	−73.84%
	BIRK/LIRK5	FMG	1	5	61	3.39×10^1	−73.53%
	LBE	SO	-	-	247	4.42×10^2	-
	LBE	FMG	1	10	60	1.34×10^2	−69.65%
4	BIRK/LIRK3	FMG	5	5	59	1.70×10^2	−61.48%
	BIRK/LIRK4	FMG	5	5	59	1.85×10^2	−57.96%
	BIRK/LIRK5	FMG	5	5	62	1.89×10^2	−57.03%
	LBE	SO	-	-	280	2.06×10^3	-
	LBE	FMG	1	5	65	6.38×10^2	−69.09%
5	BIRK/LIRK3	FMG	5	5	65	8.67×10^2	−57.97%
	BIRK/LIRK4	FMG	5	5	65	8.97×10^2	−56.55%
	BIRK/LIRK5	FMG	5	5	66	8.82×10^2	−57.24%
	LBE	SO	-	-	342	2.09×10^4	-
	LBE	FMG	1	5	66	3.90×10^3	−81.40%
6	BIRK/LIRK3	FMG	5	5	66	4.26×10^3	−79.64%
	BIRK/LIRK4	FMG	2	2	64	4.18×10^3	−80.06%
	BIRK/LIRK5	FMG	2	2	68	4.10×10^3	−80.45%

Table 1 compares the performance of the FMG algorithm coupled with the different smoothers, e.g., LBE, BIRK/LIRK3, BIRK/LIRK4, and BIRK/LIRK5, with the implicit (single-order) time integration (SO). The BIRK/LIRK smoothers show the best performance for lower solution approximations, e.g., $\mathbb{P}^l = 2, 3$, with a reduction of the computational time up to $\sim 75\%$, while the LBE smoother shows the best performance for $\mathbb{P}^l > 3$ with a reduction of the computational time up to $\sim 80\%$. For all solution approximations, the BIRK/LIRK smoothers show similar performance, and the BIRK/LIRK3 smoother is the most efficient. This is in contrast with Colombo et al. [6], where the higher number of stages guaranteed the best performance for laminar cases, while for turbulent flows the number of stages seems to have a limited influence on the computational time.

Table 2 shows the performance of the FMG algorithm with LBE and BIRK/LIRK3 smoothers for different values of pre- and post-smoothing iterations, ν_{pre} and ν_{post}. Only two choices are considered to set the smoothing iterations to avoid a wide number of simulations, i.e., $\nu_{pre} = 1$ and $\nu_{post} = \{1, 2, 5, 10, 20, 50\}$, and $\nu_{pre} = \nu_{post} = \{1, 2, 5, 10, 20, 50\}$.

A higher number of smoothing iterations decreases the number of iterations to reach the convergence, while the computational time, after reaching a minimum, starts increasing. Furthermore, the number of post-smoothing iterations, ν_{post}, has a higher influence on the performance, and $\nu_{post} > 2$ allows to avoid instabilities in problems with high stiffness. $\nu_{pre} = 1$ and $\nu_{post} = 2 - 5$ seem the

Table 2
BTC0. Performance of LBE and BIRK/LIRK3 smoothers on the coarse mesh for different post-iterations ν_{post} and $\nu_{pre} = 1$ (top) and $\nu_{pre} = \nu_{post}$ (bottom), $tol_{GMRES} = 10^{-12}$ and $CFL_{min} = 1$, $\mathbb{P}^{coarsest} = \mathbb{P}^1$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, and Δt_{CPU} the reduction due to the use of the FMG algorithms in comparison to the traditional implicit time integration.

		$\mathbb{P}^2$		$\mathbb{P}^3$		$\mathbb{P}^4$		$\mathbb{P}^5$		$\mathbb{P}^6$	
	ν_{post}	N_{ite}	t_{CPU}	N_{ite}	t_{CPU}	N_{ite}	t_{CPU}	N_{ite}	t_{CPU}	N_{ite}	t_{CPU}
LBE	1	92	1.33×10^{1}	106	5.54×10^{1}	124	2.42×10^{2}	136	8.46×10^{2}	141	2.96×10^{3}
	2	76	9.43×10^{0}	71	3.98×10^{1}	68	1.83×10^{2}	67	6.42×10^{2}	66	2.25×10^{3}
	5	65	7.46×10^{0}	64	3.84×10^{1}	64	1.50×10^{2}	64	5.25×10^{2}	64	1.84×10^{3}
	10	59	6.74×10^{0}	60	3.70×10^{1}	60	1.34×10^{2}	59	4.68×10^{2}	58	1.64×10^{3}
	20	56	7.80×10^{0}	57	4.30×10^{1}	58	1.72×10^{2}	57	6.04×10^{2}	57	2.11×10^{3}
	50	54	1.24×10^{1}	56	6.51×10^{1}	58	2.91×10^{2}	58	1.02×10^{3}	57	3.57×10^{3}
BIRK/LIRK3	1	85	1.33×10^{1}	84	9.25×10^{1}	83	5.67×10^{2}	83	3.29×10^{3}	85	1.61×10^{4}
	2	70	9.43×10^{0}	71	3.80×10^{1}	72	2.29×10^{2}	72	1.33×10^{3}	72	6.50×10^{3}
	5	60	7.46×10^{0}	61	3.39×10^{1}	62	1.89×10^{2}	63	1.10×10^{3}	62	5.38×10^{3}
	10	56	6.74×10^{0}	57	3.49×10^{1}	58	2.10×10^{2}	58	1.22×10^{3}	57	5.96×10^{3}
	20	54	7.80×10^{0}	55	4.11×10^{1}	56	2.43×10^{2}	57	1.41×10^{3}	55	6.92×10^{3}
	50	53	1.24×10^{1}	54	6.11×10^{1}	55	3.47×10^{2}	55	2.01×10^{3}	55	9.85×10^{3}

		$\mathbb{P}^2$		$\mathbb{P}^3$		$\mathbb{P}^4$		$\mathbb{P}^5$		$\mathbb{P}^6$	
	ν_{post}	N_{ite}	t_{CPU}	N_{ite}	t_{CPU}	N_{ite}	t_{CPU}	N_{ite}	t_{CPU}	N_{ite}	t_{CPU}
LBE	1	92	1.33×10^{1}	106	5.54×10^{1}	124	2.42×10^{2}	136	8.46×10^{2}	141	2.96×10^{3}
	2	69	8.80×10^{0}	68	3.65×10^{1}	69	1.94×10^{2}	67	6.79×10^{2}	66	2.38×10^{3}
	5	64	7.85×10^{0}	65	3.97×10^{1}	65	2.12×10^{2}	64	7.40×10^{2}	64	2.59×10^{3}
	10	59	8.69×10^{0}	61	4.41×10^{1}	60	2.42×10^{2}	59	8.48×10^{2}	58	2.97×10^{3}
	20	57	1.04×10^{1}	58	5.24×10^{1}	59	2.92×10^{2}	57	1.02×10^{3}	57	3.58×10^{3}
	50	55	1.68×10^{1}	57	7.48×10^{1}	58	4.31×10^{2}	58	1.51×10^{3}	57	5.28×10^{3}
BIRK/LIRK3	1	85	1.33×10^{1}	84	9.25×10^{1}	83	5.67×10^{2}	83	3.29×10^{3}	85	1.61×10^{4}
	2	68	8.80×10^{0}	69	4.39×10^{1}	70	2.74×10^{2}	71	1.59×10^{3}	70	7.80×10^{3}
	5	56	7.85×10^{0}	56	3.70×10^{1}	57	2.27×10^{2}	57	1.32×10^{3}	58	6.46×10^{3}
	10	54	8.69×10^{0}	55	4.02×10^{1}	56	2.52×10^{2}	56	1.46×10^{3}	56	7.15×10^{3}
	20	53	1.04×10^{1}	54	4.92×10^{1}	55	2.92×10^{2}	56	1.69×10^{3}	55	8.30×10^{3}
	50	53	1.68×10^{1}	53	7.68×10^{1}	54	4.16×10^{2}	54	2.41×10^{3}	55	1.18×10^{4}

best values for the LBE smoother, while for the BIRK/LIRK smoothers the optimal smoothing iterations are $\nu_{pre} = \nu_{post} = 5 - 10$. BIRK/LIRK shows a different convergence history with $\nu_{pre} = 1$ and $\nu_{pre} = \nu_{post}$, as the high numbers of pre- and post-smoothing iterations increase the performance of the FMG algorithm. The BIRK/LIRK3 smoother with $\nu_{pre} = \nu_{post}$ shows the best performance between the BIRK/LIRK smoothers for the lower number of stages; when $\nu_{pre} = 1$ a higher number of stages compensates for the lower number of smoothing iterations, and the BIRK/LIRK5 smoother shows the best performance.

The number of iterations to reach the convergence for different solution approximations is in the range $92 < N_{ite} < 141$ for the LBE smoother with $\nu_{pre} = \nu_{post} = 1$, while it is in the range $53 < N_{ite} < 55$ with $\nu_{pre} = 1$ and $\nu_{post} = 50$, or $\nu_{pre} = \nu_{post} = 50$. This means that only a higher number of pre- and post-smoothing iterations allows to satisfy the p independent condition, also known as asymptotic scalability. Ghidoni et al. [20,21,23] and Colombo et al. [6] proved that this condition was feasible only for the NS equations.

Table 3 compares the performance of the FMG algorithm with the implicit (single-order) time integration on the coarse and fine mesh. On both meshes, the automatic choice of the number of the pre- and post-smoothing iterations, ν_{pre} and ν_{post}, allows a computational time comparable or lower than values obtained with user-defined ν_{pre} and ν_{post}. Notice that α-FMG can choose different values of ν_{pre} and ν_{post} for each level, which can guarantee a further reduction of the computational time in comparison to the standard FMG algorithm.

The FMG algorithm allows to reduce the computational time up to $\sim 78\%$ on the coarse mesh and $\sim 63\%$ on the fine mesh; this reduction increases for higher solution approximations on the coarse mesh. On the contrary, on the fine mesh, the reduction decreases for higher solution approximations. The DoFs for the different solution approximations on the coarse mesh are 3.33×10^3, 8.32×10^3, 1.64×10^4, 2.91×10^4, 4.66×10^4, and 6.99×10^4, while on the fine mesh are 2.66×10^4, 6.66×10^4, 1.33×10^5, 2.33×10^5, 3.73×10^5, and 5.59×10^5. For example, the $\mathbb{P}^4$ solution on the coarse mesh has the same DoFs of the $\mathbb{P}^1$ solution on the fine mesh, and the $\mathbb{P}^6$ solution on the coarse mesh has the same DoFs of the $\mathbb{P}^2$ solution on the fine mesh. The reduction of the FMG algorithms with respect to the implicit (single-order) algorithms is higher on the coarse mesh for both cases, as the effectiveness of the FMG algorithm increases with the number of levels.

Only LBE can be used to study the influence of the coarsest level, e.g., $\mathbb{P}^0$ or $\mathbb{P}^1$, on the computational time of the FMG algorithm. Many authors in literature have spotlighted that $\mathbb{P}^{coarsest} = \mathbb{P}^0$ shows an excessive under resolution of the coarse problem, which can affect the robustness of the algorithm. Tables 1 and 2 show the performance of the FMG algorithms with the LBE and both coarsest levels. The computational time of the simulations is comparable, showing that the $\mathbb{P}^0$ solution has no influence on the performance of the FMG algorithms when LBE smoother is adopted.

Table 3
BTC0. Performance of the LBE smoother on the coarse (top) and fine (bottom) mesh, $tol_{GMRES} = 10^{-12}$, $CFL_{min} = 1$, $\mathbb{P}^{coarsest} = \mathbb{P}^0$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, and Δt_{CPU} the reduction due to the use of the FMG algorithm in comparison to the traditional implicit time integration.

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	81	4.54×10^0	-
	FMG	1	4	44	1.87×10^0	−66.11%
	a-FMG	1	5	42	1.58×10^0	−68.14%
2	SO	-	-	152	1.88×10^1	-
	FMG	1	5	43	4.66×10^0	−75.16%
	a-FMG	1	5	42	4.47×10^0	−76.16%
3	SO	-	-	216	1.28×10^2	-
	FMG	1	3	49	3.23×10^1	−74.69%
	a-FMG	1	4	46	2.94×10^1	−76.97%
4	SO	-	-	247	4.42×10^2	-
	FMG	1	10	45	1.23×10^2	−72.12%
	a-FMG	1	4	42	1.18×10^2	−73.24%
5	SO	-	-	280	2.07×10^3	-
	FMG	1	5	51	6.19×10^2	−70.03%
	a-FMG	1	3	48	5.94×10^2	−71.23%
6	SO	-	-	342	2.09×10^4	-
	FMG	1	1	53	5.01×10^3	−76.08%
	a-FMG	1	2	51	4.71×10^3	−77.52%

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	87	6.56×10^0	-
	FMG	1	2	68	2.81×10^0	−57.21%
	a-FMG	1	4	66	2.55×10^0	−61.06%
2	SO	-	-	138	4.88×10^1	-
	FMG	1	5	52	1.84×10^1	−62.32%
	a-FMG	1	3	49	1.76×10^1	−63.83%
3	SO	-	-	190	2.94×10^2	-
	FMG	1	2	54	1.23×10^2	−58.30%
	a-FMG	1	2	54	1.14×10^2	−61.22%
4	SO	-	-	249	1.94×10^3	-
	FMG	1	2	58	8.19×10^2	−57.69%
	a-FMG	1	2	56	7.78×10^2	−59.81%
5	SO	-	-	276	1.34×10^4	-
	FMG	1	2	61	5.93×10^3	−55.79%
	a-FMG	1	2	61	5.70×10^3	−57.56%
6	SO	-	-	347	7.91×10^4	-
	FMG	1	2	58	5.16×10^3	−61.53%
	a-FMG	1	2	58	5.01×10^3	−62.69%

6.2. *NASA* 65° *sweep DW*

The second test case is the NASA 65° sweep DW from the second international Vortex Flow Experiment (VFE-2) [34], at Mach number M = 0.4, Reynolds number $\mathrm{Re}_c = 3 \times 10^6$, and angle of attack of $\alpha = 13.3°$. The Reynolds number is based on the main aerodynamic chord, c, and the free-stream flow conditions, v_∞ and u_∞.

The domain is a box of dimension $15c \times 15c \times 7.5c$, where x is the stream-wise direction, y the cross-wise direction, and z the span-wise direction. A no-slip condition is applied on the body; a symmetric half-body configuration is considered for the simulations, and a symmetry condition is prescribed on the x-y plane. The mesh has 13 816 quadratic hexahedral elements, and is generated with an in-house agglomeration software, starting from the linear mesh of Crippa [36] (see Fig. 3).

Table 4 compares the performance of the FMG algorithm coupled with different smoothers, e.g., LBE, BIRK/LIRK3, BIRK/LIRK4, and BIRK/LIRK5, with the implicit (single-order) time integration. As in the BTC0 test case, the BIRK/LIRK smoothers show the best performance for lower solution approximations, e.g., $\mathbb{P}^l = 2, 3$, with a reduction of the computational time up to $\sim 80\%$, while the LBE smoother shows the best performance for $\mathbb{P}^l > 3$ with a reduction of the computational time up to $\sim 70\%$. BIRK/LIRK smoothers show similar performance for all solution approximations. In particular, the BIRK/LIRK3 smoother is the most efficient.

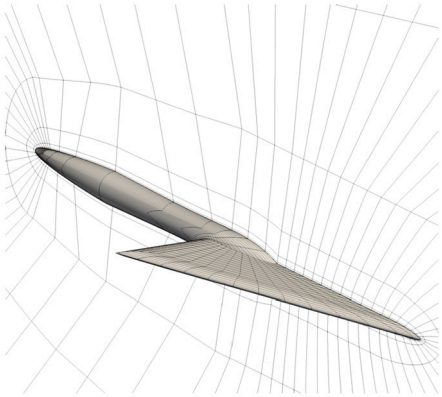


Fig. 3. NASA 65° sweep DW. Detail of the mesh on the wing and the symmetry plane, with 13 816 quadratic hexahedral elements.

Table 4
NASA 65° sweep DW. Performance of different smoothers, $tol_{GMRES} = 10^{-12}$, $CFL_{min} = 1$, $\mathbb{P}^{coarsest} = \mathbb{P}^1$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, Δt_{CPU} the reduction due to the use of the FMG algorithms in comparison to the traditional implicit time integration, and Δ_{Mem} the reduction of the memory footprint due to the use of the FMG algorithms in comparison to the traditional implicit time integration.

$\mathbb{P}^l$			ν_c	ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}	ΔMem
		SO	-	-	-	494	2.73×10^2	-	-
	LBE	FMG	5	1	5	60	5.80×10^1	−78.75%	-
2	BIRK/LIRK3	FMG	5	10	10	49	5.73×10^1	−79.01%	−86.36%
	BIRK/LIRK4	FMG	5	5	5	51	5.51×10^1	−79.82%	−86.36%
	BIRK/LIRK5	FMG	5	1	10	52	5.98×10^1	−78.10%	−86.36%
		SO	-	-	-	556	1.37×10^3	-	-
	LBE	FMG	5	1	3	65	4.45×10^2	−67.52%	-
3	BIRK/LIRK3	FMG	5	2	2	54	4.29×10^2	−68.69%	−83.96%
	BIRK/LIRK4	FMG	5	5	5	54	4.39×10^2	−67.96%	−83.96%
	BIRK/LIRK5	FMG	5	1	5	56	4.61×10^2	−66.35%	−83.96%
		SO	-	-	-	584	3.52×10^3	-	-
	LBE	FMG	5	1	5	67	1.21×10^3	−65.23%	-
4	BIRK/LIRK3	FMG	5	5	5	57	1.27×10^3	−63.91%	−83.96%
	BIRK/LIRK4	FMG	5	5	5	54	1.31×10^3	−62.88%	−83.96%
	BIRK/LIRK5	FMG	5	1	5	57	1.23×10^3	−69.94%	−83.96%
		SO	-	-	-	599	1.13×10^4	-	-
	LBE	FMG	5	1	2	68	3.23×10^3	−71.23%	-
5	BIRK/LIRK3	FMG	5	5	5	59	3.55×10^3	−68.49%	−83.96%
	BIRK/LIRK4	FMG	5	1	5	63	3.62×10^3	−67.88%	−83.96%
	BIRK/LIRK5	FMG	5	1	5	64	3.68×10^3	−67.31%	−83.96%
		SO	-	-	-	612	4.96×10^4	-	-
	LBE	FMG	5	1	2	63	1.85×10^4	−62.75%	-
6	BIRK/LIRK3	FMG	5	5	5	59	2.03×10^4	−59.02%	−83.96%
	BIRK/LIRK4	FMG	5	1	5	63	2.07×10^4	−58.27%	−83.96%
	BIRK/LIRK5	FMG	5	1	5	64	2.10×10^4	−57.53%	−83.96%

Furthermore, Table 4 compares the memory footprint of FMG algorithm (with different smoothers) with the implicit (single-order) simulations. The memory footprint of the LBE is comparable to the implicit (single-order) time integration, as the main contribution is given by the data structure associated with the highest solution approximation, while the BIRK/LIRK smoothers show a reduction of the memory footprint around 85%.

Table 5 shows the performance of the FMG algorithm in comparison with the implicit (single-order) time integration. As in the BTC0 test case, the automatic choice of the number of the pre- and post-smoothing iterations, ν_{pre} and ν_{post}, allows a computational time comparable or lower than values with the user-defined ν_{pre} and ν_{post}. The FMG algorithm allows to reduce the computational time up to $\sim 83\%$ with $tol_{GMRES} = 10^{-12}$ and up to $\sim 92\%$ with $tol_{GMRES} = 10^{-2}$; the reduction decreases for higher solution approximations.

In fact, another important parameter that must be set is the absolute/relative tolerance of the GMRES. Table 5 shows the performance of the FMG algorithm with different relative tolerances of the GMRES, i.e., $tol_{GMRES} = 10^{-12}$ and $tol_{GMRES} = 10^{-2}$. The higher relative tolerance of the GMRES, i.e., $tol_{GMRES} = 10^{-2}$, allows to reduce the computational cost of the implicit (single-order)

Table 5
NASA 65° sweep DW. Performance of the LBE smoother and influence of the relative tolerance of the GMRES, $tol_{GMRES} = 10^{-12}$ (top) and $tol_{GMRES} = 10^{-2}$ (bottom), $CFL_{min} = 1$ and $\mathbb{P}^{coarsest} = \mathbb{P}^0$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, and Δt_{CPU} the reduction due to the use of the FMG algorithm in comparison to the traditional implicit time integration.

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
	SO	-	-	429	1.13×10^2	-
1	FMG	1	5	39	2.04×10^1	−81.95%
	a-FMG	1	6	37	1.90×10^1	−83.21%
	SO	-	-	494	2.73×10^2	-
2	FMG	1	5	57	5.90×10^1	−78.39%
	a-FMG	1	4	57	5.37×10^1	−80.33%
	SO	-	-	556	1.37×10^3	-
3	FMG	1	3	65	4.65×10^2	−66.14%
	a-FMG	1	3	62	4.37×10^2	−68.09%
	SO	-	-	584	3.52×10^3	-
4	FMG	1	3	98	1.23×10^3	−65.06%
	a-FMG	1	2	97	1.16×10^3	−67.15%
	SO	-	-	599	1.13×10^4	-
5	FMG	1	2	111	3.48×10^3	−69.12%
	a-FMG	1	2	108	3.34×10^3	−70.35%
	SO	-	-	612	4.96×10^4	-
6	FMG	1	2	115	1.90×10^4	−61.64%
	a-FMG	1	2	114	1.86×10^4	−62.41%

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
	SO	-	-	463	9.55×10^1	-
1	FMG	1	10	53	7.79×10^0	−91.84%
	a-FMG	1	8	50	7.71×10^0	−92.50%
	SO	-	-	553	2.30×10^2	-
2	FMG	1	5	55	2.88×10^1	−87.48%
	a-FMG	1	4	54	2.74×10^1	−88.10%
	SO	-	-	693	1.99×10^3	-
3	FMG	1	2	80	2.95×10^2	−85.18%
	a-FMG	1	3	80	2.68×10^2	−86.51%
	SO	-	-	743	4.98×10^3	-
4	FMG	1	2	97	6.89×10^2	−85.18%
	a-FMG	1	3	95	6.68×10^2	−86.58%
	SO	-	-	799	1.13×10^4	-
5	FMG	1	2	109	2.18×10^3	−80.66%
	a-FMG	1	2	108	2.11×10^3	−81.24%
	SO	-	-	856	7.68×10^4	-
6	FMG	1	2	112	1.10×10^4	−85.98%
	a-FMG	1	2	112	1.05×10^4	−86.69%

time integration of $\sim 10\%$ for lower solution approximations, i.e., between $\mathbb{P}^0$ and $\mathbb{P}^2$, while introduces convergence instabilities for higher solution approximations that increase the number of iterations to reach the convergence, and the corresponding computational cost. Nevertheless, the FMG algorithm allows to reduce the computational cost in comparison to the implicit (single-order) time integration, and the reduction increases by decreasing the tolerance of the GMRES. In fact, the computational time reduces between 81.95% and 61.61% for $tol_{GMRES} = 10^{-12}$, and between 91.84% and 85.98% for $tol_{GMRES} = 10^{-2}$, as the solution approximation is increased. In general, the number of iterations to reach the convergence increases, but the computational time per iteration decreases with a lower relative tolerance of the GMRES. Even if the results show a better performance for multigrid increasing GMRES tolerance, this "effect" is case dependent, i.e., for more complex testcases also *p*-MG shows a degradation of the performance, and ad hoc GMRES tolerances should be selected. For this reason, in Secs. 7 and 8 only $tol_{GMRES} = 10^{-12}$ has been considered on the finest level of the *p*-MG algorithm.

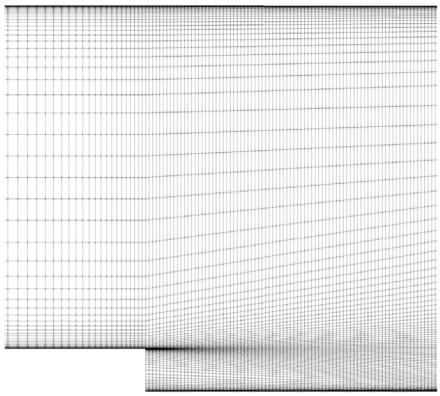


Fig. 4. BFS. Detail of the mesh near the step, with 20 540 linear hexahedral elements.

7. Verification cases

The initial assessment shown in Sec. 6 suggests the use of the LBE as smoother of the FMG to exploit the capability to use $\mathbb{P}^0$ solution approximation as the coarsest level, which can be considered of paramount importance when dealing with large meshes. In the following, only the LBE smoother and *a*-FMG algorithm will be considered. The *p*-MG performance is compared with single order simulation on the following verification cases, and predicted results are compared with experimental data:

(i) the compressible turbulent flow through the quasi-three-dimensional Backward Facing Step (BFS) from the experiments of Vogel and Eaton [37] at Mach number $\mathrm{M} = 0.128$ and Reynolds number $\mathrm{Re}_h = 28\,000$, based on the step height, h, and the inlet bulk velocity,

(ii) the transonic flow around a RAE2822 airfoil from the NPARC Alliance Validation Archive [38] of NASA at Mach number $\mathrm{M} = 0.8$ and Reynolds number $\mathrm{Re}_c = 6.5 \times 10^6$, based on the chord of the airfoil, c, and the free-stream flow conditions.

The maximum polynomial number of the solution is $\mathbb{P}^3$, and the coarsest level is $\mathbb{P}^{coarsest} = \mathbb{P}^0$. Only the *a*-FMG with the LBE smoother is used and compared with implicit single order simulations.

7.1. BFS

The first test case is the turbulent flow over a BFS, characterized by the same conditions of the experiments of Vogel and Eaton [37], i.e., Mach number $\mathrm{M} = 0.128$ and Reynolds number $\mathrm{Re}_h = 28\,000$, based on the step height, h, and the inlet bulk velocity, u_{bulk}. This case is used for the validation and assessment of turbulence models. In fact, the BFS is a validation case of the Langley Research Center Turbulence Modeling Resource of NASA [39], where experiments of Driver and Seegmiller [40] are available at $\mathrm{Re}_h = 36\,000$, which is slightly higher than the Reynolds number of Vogel and Eaton [37]. This test case, characterized by the separation and reattachment of the flow downstream the step, allows to validate turbulence models for RANS, SubGrid Scale (SGS) models for LES, and shielding functions of the hybrid RANS-LES models.

In the stream-wise direction, x, the domain ranges from $-130h$ to $50h$, and the step is located at $x = 0$; the height of the channel, i.e., the dimension in the cross-wise direction y, is $8h$ at the inlet and $9h$ at the outlet. The length of the upstream channel allows to have a fully developed turbulent boundary layer and to match the experimental flow conditions at the step. The mesh is generated according to the case description of the Langley Research Center Turbulence Modeling Resource of NASA [39], even if the Reynolds number is higher. The mesh has 20 540 hexahedral elements with linear edges with only one element in the span-wise direction, z, i.e., the domain is quasi-three-dimensional (Fig. 4). A no-slip boundary condition is applied to the upper and lower walls, and a symmetry boundary condition is applied upstream the channel from $-130h$ to $-110h$.

Fig. 5 shows the predicted skin friction coefficient, c_f, distribution on the lower wall and the non-dimensional stream-wise velocity, u_x/u_{ref}, profiles in the wall-normal or cross-wise direction at different stream-wise locations downstream the step, e.g., $x/h = \{2.2, 3.0, 3.7, 4.5, 5.2, 5.9, 6.5\}$, which are compared with the experiments of Vogel and Eaton [37], and the numerical results of Spalart et al. [41] and Travin et al. [42]. The skin friction coefficient is calculated as $c_f = (\tau_w)_x/(1/2\rho_{ref}u_{ref}^2)$, where $(\tau_w)_x$ is the stream-wise component of the wall shear stress and ρ_{ref} and u_{ref} are the density and velocity on the horizontal axis in the center of the domain at $x/h = 3.0$. The simulation with the FMG algorithm and $\mathbb{P}^3$ solution approximation is in agreement with the references, without an underestimation of the skin friction coefficient in the attached turbulent boundary layer on the lower wall.

Table 6 compares the performance of the FMG algorithm with the implicit (single-order) time integration. The FMG algorithms allow to reduce the computational time up to $\sim 82\%$, even if the reduction decreases for higher solution approximations.

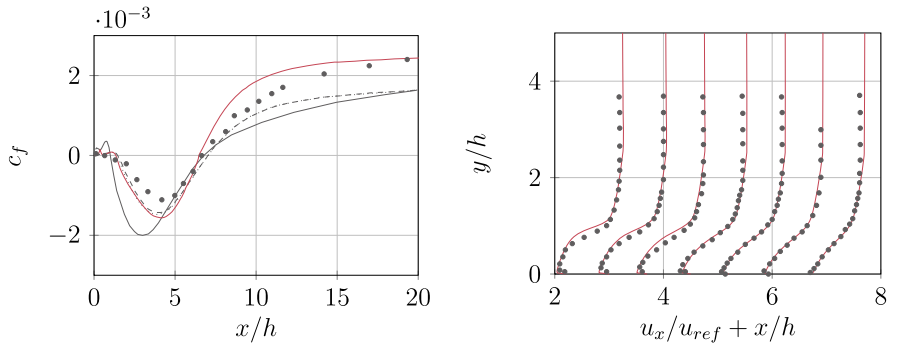


Fig. 5. BFS. Span-wise-averaged skin friction coefficient, c_f, distribution on the lower wall and non-dimensional stream-wise velocity, u_x/u_{ref}, profiles in the cross-wise direction at different stream-wise location downstream the step, e.g., $x/h = \{2.2, 3.0, 3.7, 4.5, 5.2, 5.9, 6.5\}$. The results of Spalart et al. [41] are with the SA turbulence model, while the results of Travin et al. [42] are with the k-ω SST turbulence model and two- and three-dimensional domains. • Vogel and Eaton exp. [37], —— Spalart et al. SA [41], – – – Travin et al. 3D k-ω SST [42], ····· Travin et al. 2D k-ω SST [42], —— $\mathbb{P}^3$ solution.

Table 6
BFS. Performance of the a-FMG algorithm with LBE smoother, $tol_{GMRES} = 10^{-12}$, $CFL_{min} = 1$, $\mathbb{P}^{coarsest} = \mathbb{P}^0$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, and Δt_{CPU} the reduction due to the use of the FMG algorithm in comparison to the traditional implicit time integration.

$\mathbb{P}^l$		ν_c	ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	-	6517	6.58×10^2	-
	a-FMG	5	1	10	6374	1.24×10^2	−81.22%
2	SO	-	-	-	7736	4.19×10^3	-
	a-FMG	5	1	5	6694	8.21×10^2	−80.42%
3	SO	-	-	-	8454	9.60×10^3	-
	a-FMG	5	1	2	6719	2.78×10^3	−71.03%

7.2. Transonic RAE2822 airfoil

The second test case is the turbulent and transonic flow around the RAE2822 airfoil from the AGARD report of the experiment of Cook et al. [43], at Mach number M = 0.725, Reynolds number $\mathrm{Re}_c = 6.5 \times 10^6$, and angle of attack of $\alpha = 2.92°$. The Reynolds number is based on the chord of the airfoil, c, and the free-stream flow conditions, ν_∞ and u_∞. This case is considered to study the performance of the FMG algorithm with a flow field characterized by a shock, and with a mesh that presents a refinement along the shock location and a high AR of the elements.

This airfoil is a case of the NPARC Alliance Validation Archive of NASA [38]. The accuracy of RANS and different turbulence models has been assessed in terms of pressure, c_p, and skin friction, c_f, coefficients on the airfoil. Nelson [38] spotlighted that different turbulence models, e.g., Cebeci-Smith, Baldwin-Lomax, SA, k-ω, and k-ϵ have a good agreement with the experiments. The turbulence models differ only in the location of the shock, but the difference between the numerical results and the experiments is negligible.

The domain is a circular cylinder with radius $20c$, where x is the stream-wise direction, y the cross-wise direction, and z the span-wise direction. The mesh has 8 096 hexahedral elements with quadratic edges with only one element in the span-wise direction, i.e., the domain is quasi-three-dimensional (Fig. 6). The mesh shows a refinement in the shock and wake regions; the shock location is at $x/c = 0.52$, as measured by experiments. A no-slip condition is applied to the airfoil, and a far-field condition is applied to the external boundary of the domain.

Fig. 7 compares the predicted pressure, c_p, and skin friction, c_f, coefficient distributions on the airfoil with the experiments of Cook et al. [43], and the numerical results of the NPARC Alliance Validation Archive of NASA [38]. The pressure and skin friction coefficients are calculated as $c_p = (p - p_\infty)/(1/2\rho_\infty u_\infty^2)$ and $c_f = (\tau_w)_x/(1/2\rho_\infty u_\infty^2)$. The predicted results with $\mathbb{P}^3$ solution approximation are in agreement with the references. Fig. 6 (left) show also the corresponding Mach number contours.

Table 7 compares the performance of the FMG algorithm with the implicit (single-order) time integration. The transonic cases are characterized by a sharp solution in the shock flow region, where it is well known that the performance of h- and p-MG algorithms degrades with non-smooth solutions due to shocks or high AR of the elements of the mesh [5,24]. The a-FMG algorithm allows to reduce the computational time up to ∼ 77%, and the reduction only slightly decreases for higher solution approximations. The p-MG algorithm does not show any robustness issue with the shock.

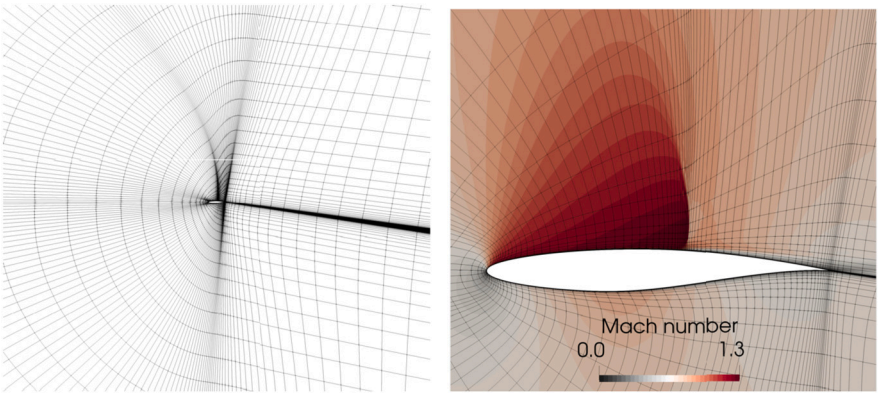


Fig. 6. RAE2822 airfoil. Mesh (left) of the computational domain (8 096 quadratic hexahedral elements), and Mach number contours (right), $\mathbb{P}^3$ solution.

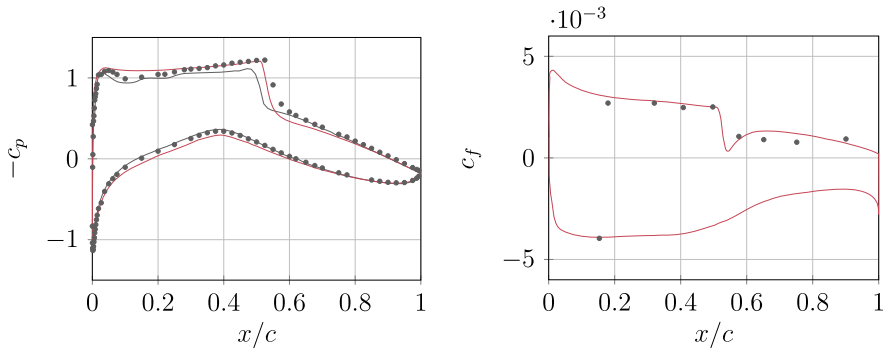


Fig. 7. RAE2822 airfoil. Span-wise-averaged pressure, $-c_p$, and skin friction, c_f, coefficients on the airfoil. The skin friction coefficient distribution on the lower side of the airfoil is reported with the minus. • Cook et al. exp. [43], —— Nelson k-ω [38], —— $\mathbb{P}^3$ solution.

Table 7
RAE2822 airfoil. Performance of the a-FMG algorithm with LBE smoother, $tol_{GMRES} = 10^{-12}$, $CFL_{min} = 1$, $\mathbb{P}^{coarsest} = \mathbb{P}^0$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, and Δt_{CPU} the reduction due to the use of the FMG algorithm in comparison to the traditional implicit time integration.

$\mathbb{P}^l$		ν_{coarse}	ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	-	4323	2.18×10^2	-
	a-FMG	5	1	5	3053	5.56×10^1	−74.56%
2	SO	-	-	-	5578	1.38×10^3	-
	a-FMG	5	1	3	3106	3.17×10^2	−77.05%
3	SO	-	-	-	6018	4.45×10^3	-
	a-FMG	5	1	2	3338	1.44×10^3	−67.57%

8. Industrial cases

Finally, the effectiveness and robustness of the proposed p-MG algorithm is demonstrated in the computation of complex industrial problems:

(i) the subsonic and transonic flow through the low pressure turbine cascade T106A, designed by MTU Aero Engines for subsonic (Reynolds number $\mathrm{Re}_{2,is} = 0.5 \times 10^6$ and outlet isentropic Mach number $\mathrm{M}_{2,is} = 0.59$) and transonic (Reynolds number $\mathrm{Re}_{2,is} = 0.5 \times 10^6$ and outlet isentropic Mach number $\mathrm{M}_{2,is} = 0.955$) operating conditions,
(ii) the turbulent flow around the simplified fastback DrivAer car from the experiments of Heft et al. [44] at Mach number $\mathrm{M} = 0.1$ and Reynolds number $\mathrm{Re}_L = 4.87 \times 10^6$, based on the length of the car, L, and free-stream flow conditions,
(iii) the turbulent subsonic flow around the aircraft NASA Trap-Wing from the 1st AIAA CFD High Lift Prediction Workshop (HiLift-1) [45] at Mach number $\mathrm{M} = 0.1$, Reynolds number $\mathrm{Re}_c = 4.3 \times 10^6$, and an angle of attack of $\alpha = 13°$,

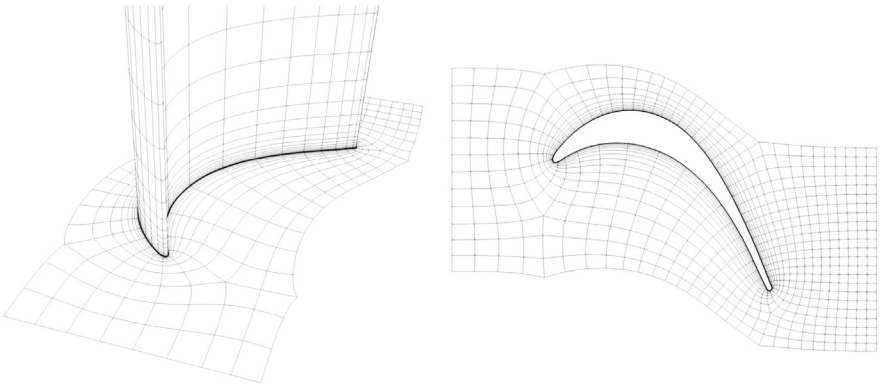


Fig. 8. T106A. Detail of the three-dimensional coarse mesh on the blade and an endwall (left) and quasi-three-dimensional coarse mesh (right), with 43 200 and 1 271 quadratic hexahedral elements.

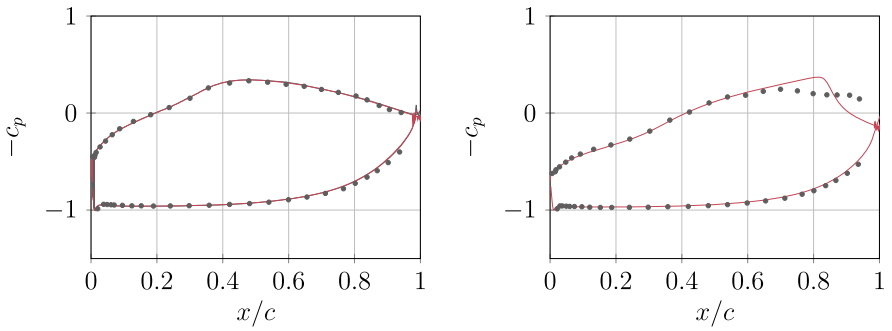


Fig. 9. T106A. Pressure coefficient, $-c_p$, distribution on the blade at mid-span in the subsonic (left) and transonic (right) case with the three-dimensional fine mesh. The results of Lorini et al. [48,49] are with k_L-k-$\tilde{\omega}$ transition model and the same mesh and solution approximation. • Hoheisel exp. [47], —— Lorini et al. k_L-k-$\tilde{\omega}$ [48,49], —— $\mathbb{P}^3$ solution.

(*iv*) the turbulent transonic flow around the aircraft DLR-F6 drom the 3*rd* AIAA CFD Drag Prediction Workshop [46] at Mach number $M = 0.75$ and Reynolds number $Re_c = 5 \times 10^6$, based on the main aerodynamic chord, c, and free-stream flow conditions.

The maximum polynomial number of the solution is $\mathbb{P}^3$, and the coarsest level is $\mathbb{P}^{coarsest} = \mathbb{P}^0$. As in Sec. 7, only the a-FMG with the LBE smoother is used and compared with implicit single order simulations.

8.1. T106A

The first test case is the T106A turbine cascade, designed by MTU Aero Engine, for subsonic and transonic operating conditions. In the subsonic case the Reynolds number is $Re_{2,is} = 0.5 \times 10^6$ and the outlet isentropic Mach number is $M_{2,is} = 0.59$, while in the transonic case the Reynolds number is $Re_{2,is} = 0.5 \times 10^6$ and the outlet isentropic Mach number is $M_{2,is} = 0.955$. Both cases have an inlet turbulence intensity of $Tu_1 = 4.0\%$, and an inlet flow angle of $\alpha_1 = 37.7°$. The Reynolds number is based on the downstream isentropic conditions, and the blade chord, c. Simulations of both operating conditions are carried out on two different meshes, corresponding to a quasi-three- (section at midspan) and a three-dimensional (full geometry with end walls) domain. The three-dimensional meshes have 43 200 and 77 300 quadratic hexahedral elements, while the quasi-three-dimensional meshes have 1 271 and 4 626 quadratic hexahedral elements (see Fig. 8).

This case is used in literature as validation case for turbulence and transition models, as it shows different transition mechanisms for different Reynolds numbers, i.e., separation-induced (SI) transition at $Re_{2,is} = 0.15 \times 10^6$, bypass (BP) transition at $Re_{2,is} = 0.5 \times 10^6$ and $Re_{2,is} = 1.1 \times 10^6$. In particular, at $Re_{2,is} = 0.5 \times 10^6$ the experiments of Hoheisel [47] are available for different downstream isentropic Mach numbers, $M_{2,is}$, e.g., 0.3, 0.59, 0.8, 0.9 and 0.955. In the transonic case the shock promotes the separation of the flow in the second part of the suction side, and the transition happens in the shear layer.

Fig. 9 compares the pressure coefficient, $-c_p$, distribution on the blade at mid-span with experiments of Hoheisel [47] for both subsonic and transonic cases. The pressure coefficient is calculated as $c_p = (p_{01} - p)/(p_{01} - p_2)$, where p and p_0 are the static and total pressure, while 1 and 2 refer to the inlet and outlet boundaries, respectively. Furthermore, Fig. 10 shows the Mach number contours at mid-span for both operating conditions on the three-dimensional fine mesh.

For the subsonic case, Table 8 compares the performance of the a-FMG algorithm with the LBE smoother with the implicit (single-order) time integration on the coarse and fine mesh for both domains. On the quasi-three-dimensional coarse mesh, the FMG algorithm allows to reduce the computational time up to $\sim 90\%$ with the $\mathbb{P}^3$ solution, while on the three-dimensional fine mesh, up to

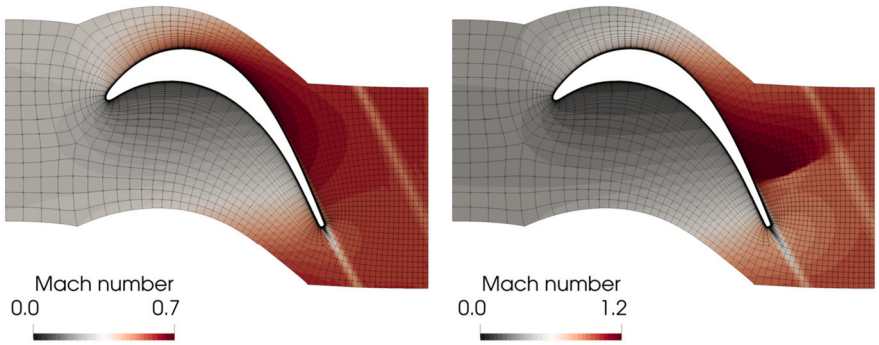


Fig. 10. T106A. Mach number, M, contours at mid-span in the subsonic (left) and transonic (right) case on the three-dimensional fine mesh, $\mathbb{P}^3$ solution.

Table 8
T106A. Performance of the a-FMG algorithm with LBE smoother on the coarse (left) and fine (right) mesh for the quasi-three-dimensional (top) and three-dimensional (bottom) domain, subsonic case, $tol_{GMRES} = 10^{-12}$, $CFL_{min} = 1$, $\mathbb{P}^{coarsest} = \mathbb{P}^0$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, and Δt_{CPU} the reduction due to the use of the FMG algorithm in comparison to the traditional implicit time integration.

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	285	1.27×10^1	-
	a-FMG	1	2	111	5.30×10^0	−58.27%
2	SO	-	-	432	5.18×10^1	-
	a-FMG	1	2	74	8.69×10^0	−83.23%
3	SO	-	-	755	4.45×10^2	-
	a-FMG	1	2	68	3.82×10^1	−91.41%

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	414	4.23×10^1	-
	a-FMG	1	3	129	8.01×10^0	−81.06%
2	SO	-	-	529	1.28×10^2	-
	a-FMG	1	2	118	3.13×10^1	−75.55%
3	SO	-	-	606	4.41×10^2	-
	a-FMG	1	2	149	1.96×10^2	−55.56%

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	2467	8.87×10^2	-
	a-FMG	1	2	998	2.48×10^2	−72.01%
2	SO	-	-	2785	2.42×10^3	-
	a-FMG	1	4	804	8.88×10^2	−63.31%
3	SO	-	-	3009	9.22×10^3	-
	a-FMG	1	4	859	4.22×10^3	−54.22%

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	3584	2.96×10^3	-
	a-FMG	1	4	1184	2.67×10^2	−90.99%
2	SO	-	-	3410	6.00×10^3	-
	a-FMG	1	2	1311	2.19×10^3	−63.58%
3	SO	-	-	2415	9.14×10^3	-
	a-FMG	1	2	1889	4.29×10^3	−53.09%

$\sim 90\%$ with the $\mathbb{P}^1$ solution. On the quasi-three-dimensional coarse mesh, the performance of the FMG algorithm increases for higher solution approximations, while on the three-dimensional fine mesh, the performance of the FMG algorithm decreases by increasing the solution approximation. However, the worst reduction of the computational time with respect to the implicit (single-order) time integration guarantees a saving of about 49%.

In all the simulations, the residual of the logarithm of the specific dissipation rate, $\widetilde{\omega}$, decreases last to the convergence. At the moment of the convergence on the quasi-three-dimensional coarse mesh and $\mathbb{P}^3$ solution approximation, the reduction of the computational time in comparison with the implicit (single-order) time integration is 90.76%. An even higher reduction can be obtained in the previous iterations. For example, the computational time reduces to 95.01% with a L^2 norm of the residual lower than $\sim 10^{-3}$. These values demonstrate how the FMG algorithms guarantee a fast reduction of the residuals, and a quasi-convergence is available even in the first iterations of the FAS between the coarsest and finest levels.

For the transonic case, Table 9 compares the performance of the a-FMG algorithm with the LBE smoother with the implicit (single-order) time integration on the coarse and fine mesh of the quasi-three-dimensional and three-dimensional domain. The FMG algorithm guarantees a reduction of the computational time up to $\sim 95\%$ for all domains, meshes, and solution approximations. As in the RAE2822 airfoil case, the FMG algorithm does not show any reduction of the performance for increasing solution approximations.

8.2. DrivAer car

The turbulent flow over a simplified fastback DrivAer car from the experiments of Heft et al. [44] is here considered. In particular, the handles, the rear-view mirrors, and the wheels are removed, the underbody is smooth, and the front radiator grid is closed. The Reynolds number is $\mathrm{Re}_L = 4.87 \times 10^6$, based on the model length, $L = 4.6\,m$, and the free-stream flow conditions, ν_∞ and u_∞. The free-stream turbulence intensity is $Tu = 0.1\%$, and the Mach number is set to $\mathrm{M} = 0.1$ to consider the flow essentially incompressible.

Since the development of the DrivAer car in the different configurations, e.g., fastback, notchback and squareback, many works studied the accuracy of turbulence models, e.g., RANS, LES, and hybrid RANS-LES models, and numerical schemes in the description of the flow and acoustic phenomena around the car. The DrivAer car, as well as the Ahmed body, can be considered a reference for the scientific community interested in the simulation of the flow around vehicles. Heft et al. [44] compared the experiments and

Table 9
T106A. Performance of the a-FMG algorithm with LBE smoother on the coarse (left) and fine (right) mesh for the quasi-three-dimensional (top) and three-dimensional (bottom) domain, transonic case, $tol_{GMRES} = 10^{-12}$, $CFL_{min} = 1$, $\mathbb{P}^{coarsest} = \mathbb{P}^0$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, and Δt_{CPU} the reduction due to the use of the FMG algorithm in comparison to the traditional implicit time integration.

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	573	2.55×10^1	-
	a-FMG	1	2	257	4.37×10^0	−82.87%
2	SO	-	-	867	1.15×10^2	-
	a-FMG	1	2	387	1.97×10^1	−82.86%
3	SO	-	-	1719	1.44×10^3	-
	a-FMG	1	1	298	1.91×10^2	−86.73%

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	19389	1.43×10^3	-
	a-FMG	1	2	696	1.16×10^2	−91.90%
2	SO	-	-	20794	3.48×10^3	-
	a-FMG	1	2	554	1.57×10^2	−93.14%
3	SO	-	-	22377	9.07×10^3	-
	a-FMG	1	1	603	8.15×10^2	−91.02%

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	4960	1.78×10^3	-
	a-FMG	1	2	2330	3.14×10^2	−82.35%
2	SO	-	-	5589	5.38×10^3	-
	a-FMG	1	4	4122	1.26×10^3	−76.56%
3	SO	-	-	6851	2.98×10^4	-
	a-FMG	1	4	3621	4.05×10^3	−86.40%

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	167834	9.99×10^4	-
	a-FMG	1	2	6105	6.62×10^3	−93.38%
2	SO	-	-	134054	1.37×10^5	-
	a-FMG	1	4	5801	1.07×10^4	−90.03%
3	SO	-	-	89182	1.88×10^5	-
	a-FMG	1	4	7216	1.20×10^4	−93.61%

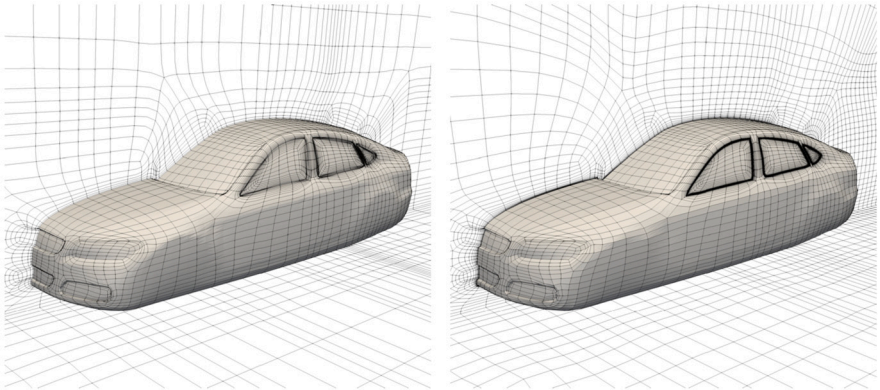


Fig. 11. DrivAer car. Detail of the coarse (left) and fine (right) mesh on the car and the symmetry plane, with 48 264 and 113 780 cubic hexahedral elements.

simulations with the RANS equations with different turbulence models, e.g., k-ω SST and Scale-Adaptive Simulation (SAS), in terms of the skin friction coefficient, c_f, distribution on the car. Furthermore, Rüttgers et al. [50], Colombo et al. [51], and Ashton and Revell [52] carried out high-fidelity simulations, with LES or hybrid RANS-LES models, to describe the separation and reattachment points of the flow on the car, the strength and unsteadiness of the vortices. In particular, Colombo et al. [51] used a p-MG algorithm as multilevel preconditioner for the linear solver of the implicit (single-order) time integration for an Implicit LES (ILES) of the simplified fastback DrivAer car.

The domain is a box of dimension $12L \times 3L \times 3L$, where x is the stream-wise direction, y the span-wise direction, and z the direction normal to the ground. The inlet boundary is at $3L$ from the frontal stagnation point, while the outlet boundary at $8L$ from the rear part of the car. The cross-wise blockage ratio is higher than the critical value, i.e., $BR > 8$. No-slip boundary conditions are applied to the car and the ground, which is moving at the same velocity of the free-stream flow u_∞. According to Rüttgers et al. [50], a symmetric half-body configuration is considered for the simulations, and a symmetry condition is prescribed on the x-z plane. Different meshes are considered, generated with the software GridPro [53] and kindly shared by Prof. Alessandro Colombo of Università degli studi di Bergamo [51], consisting of 48 264 and 113 780 cubic hexahedral elements (see Fig. 11). The corresponding DoFs for the $\mathbb{P}^3$ solution approximation are 965 280 and 2.27×10^6 DoFs, respectively, which are lower than the mesh elements used for RANS simulations by Heft et al. [44] (19.4×10^6 elements), and Ashton and Revell [52] (18, 37 and 80×10^6 elements).

Fig. 12 compares the predicted pressure coefficient, c_p, distribution on the upper and lower part of the car at symmetry plane with the experiments and the RANS simulations of Heft et al. [44]. Furthermore, Fig. 13 shows the pressure coefficient, c_p, distribution on the car, the ground, and the symmetry plane, and the turbulent intensity, Tu, contours on different slices behind the car. The pressure coefficient is calculated as $c_p = (p - p_\infty)/(1/2\rho_\infty u_\infty^2)$, and the turbulent intensity as $Tu = \sqrt{2/3k}/u_\infty$, where k is the turbulent kinetic energy. The simulations with the a-FMG algorithm and $\mathbb{P}^3$ solution approximation shows a higher agreement with the experiments than the RANS results of Heft et al. [44].

Table 10 compares the performance of the a-FMG algorithm with LBE smoother with the implicit (single-order) time integration for the coarse and fine mesh. The FMG algorithm allows to reduce the computational time up to ∼ 70% on the coarse mesh, and up

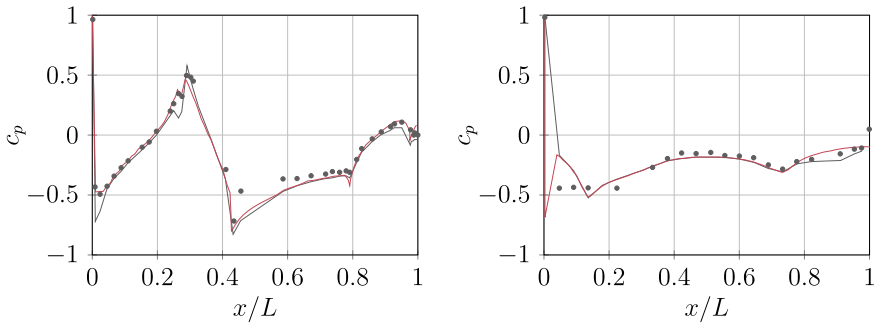


Fig. 12. DrivAer car. Pressure coefficient, c_p, distribution on the symmetry plane and the upper (left) and lower (right) part of the car with the fine mesh. • Heft et al. exp. [44], —— Heft et al. k-ω [44], —— $\mathbb{P}^3$ solution.

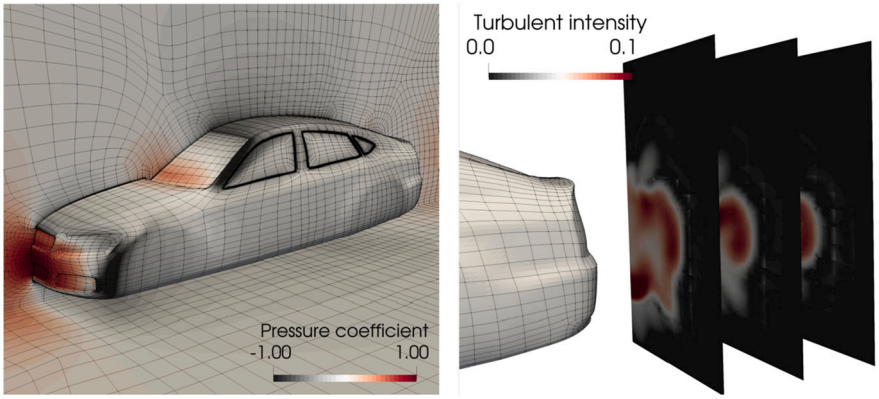


Fig. 13. DrivAer car. Pressure coefficient, c_p, distribution on the car (left) and turbulent intensity, $Tu = \sqrt{2/3k}/u_\infty$, contours on different slices behind the car (right) with the fine mesh, $\mathbb{P}^3$ solution.

Table 10
DrivAer car.Performance of the a-FMG algorithm with LBE smoother on coarse (top) and fine (bottom) mesh, $tol_{GMRES} = 10^{-12}$, $CFL_{min} = 1$, $\mathbb{P}^{coarsest} = \mathbb{P}^0$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, and Δt_{CPU} the reduction due to the use of the FMG algorithm in comparison to the traditional implicit time integration.

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	335	1.35×10^3	-
	a-FMG	1	5	124	3.95×10^2	−70.73%
2	SO	-	-	856	3.69×10^3	-
	a-FMG	1	3	126	1.52×10^3	−58.73%
3	SO	-	-	998	1.38×10^4	-
	a-FMG	1	2	132	5.78×10^3	−55.55%

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	613	2.84×10^3	-
	a-FMG	1	3	137	1.12×10^3	−60.46%
2	SO	-	-	817	9.59×10^3	-
	a-FMG	1	2	135	4.69×10^3	−51.10%
3	SO	-	-	899	5.33×10^4	-
	a-FMG	1	2	146	2.61×10^4	−51.04%

to $\sim 60\%$ on the fine mesh, even if the performance decreases for higher solution approximations on both meshes. Notice that the performance of the p-MG algorithm seems to be only DoFs dependent. For example, the $\mathbb{P}^2$ (4.83×10^5) and $\mathbb{P}^3$ (9.65×10^5) solutions on the coarse mesh have the same DoFs and reduction of the computational time shown by the $\mathbb{P}^1$ (4.55×10^5) and $\mathbb{P}^2$ (1.14×10^6) solutions on the fine mesh.

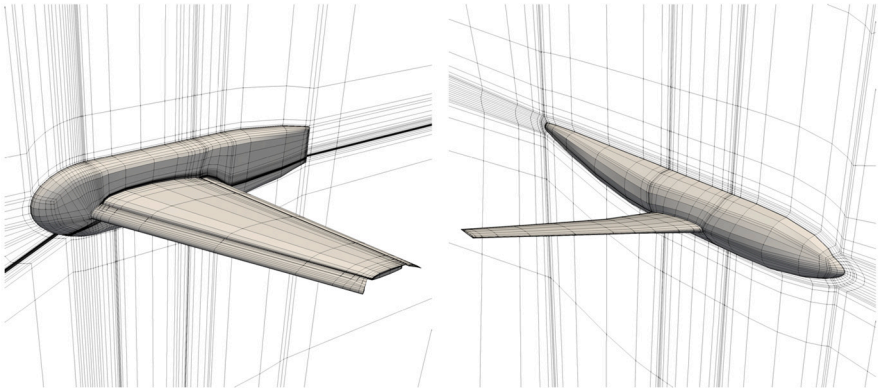


Fig. 14. NASA's workshop aircraft. Detail of the meshes on the aircraft and the symmetry plane for the NASA Trap-Wing (left) and the DLR-F6 (right), with 93 088 and 50 618 cubic hexahedral elements.

8.3. NASA's workshops aircraft

The last test cases are the turbulent flow around the aircraft NASA Trap-Wing from the 1*st* AIAA CFD High Lift Prediction Workshop (HiLift-1) [45] at Mach number M = 0.1, Reynolds number $\mathrm{Re}_c = 4.3 \times 10^6$, and angle of attack of $\alpha = 13°$, and the turbulent and transonic flow around the aircraft DLR-F6 from the 3*rd* AIAA CFD Drag Prediction Workshop (DPW3) [46] at Mach number M = 0.75 and Reynolds number $\mathrm{Re}_c = 5 \times 10^6$. The Reynolds number is based on the main aerodynamic chord, c, and the free-stream flow conditions, v_∞ and u_∞. Experiments from the NASA Langley Research Center and the NASA Ames Research Center are available only for the NASA Trap-Wing at the same Reynolds and Mach numbers, and the data are corrected to consider free flight conditions. Experiments for the DLR-F6 are available for the geometry with the nacelle, and a slightly lower Reynolds number, $\mathrm{Re}_c = 3 \times 10^6$.

The NASA Trap-Wing has the following geometrical characteristics: mean aerodynamic chord $c = 1.06\,m$ (MAC), half span $2.12\,m$, AR = 4.56, leading edge sweep of 33.9°, quarter chord sweep of 30°, and Taper Ratio $TR = 0.4$. In the landing configuration, the slat and the flap have angles of attack of 30° and 25°, respectively. This aircraft is designed for high-lift conditions. The DLR-F6 represents an Airbus aircraft designed for transonic conditions with an AR of the wings of 9.5, a leading-edge sweep of 27.1°, and a quarter-chord sweep of 25°. This aircraft is characterized by a dihedral of the wing of 4.787°, and an acute angle of $\sim 60°$ between the fuselage and the wing upper-surface at the trailing-edge, which leads to a separation of the flow.

The domains have the following dimensions: $240c \times 240c \times 120c$ for the NASA Trap-Wing, and $100c \times 100c \times 50c$ for the DLR-F6. x is the stream-wise direction, y the cross-wise direction, and z the span-wise direction. A no-slip boundary condition is applied on the aircraft. A symmetric half-body configuration is considered for the simulations, and a symmetry condition is prescribed on the x-y plane.

The linear multiblock structured meshes for both aircraft [45,46] are used to generate high-order meshes with an in-house agglomeration software. In particular, the mesh for the NASA Trap-Wing has 93 088 quadratic hexahedral elements, while the DLR-F6 mesh has 50 618 cubic hexahedral elements (see Fig. 14).

Fig. 15 compares the predicted lift coefficient, c_L, for different angles of attack for both aircraft with experiments [45] and numerical results [46]. Fig. 16 shows the pressure coefficient, c_p, distribution on the fuselage and wing of both aircraft with a $\mathbb{P}^3$ solution approximation. The pressure coefficient is calculated as $c_p = (p - p_\infty)/(1/2\rho_\infty u_\infty^2)$ and the lift coefficient as $c_L = L/(1/2\rho_\infty u_\infty^2 A_{ref})$, where $L = F_y$ is the cross-wise component of the force applied to the aircraft, and A_{ref} is the wing section. Simulations are carried out at $\alpha = \{13°, 15°, 20°, 25°, 30°, 35°, 38°\}$ for the NASA Trap-Wing and $\alpha = \{-3°, -2°, -1°, -0.5°, 0°, 0.5°, 1°, 1.5°\}$ for the DLR-F6. The comparison of the performance between *a*-FMG algorithm and implicit single order simulations is carried out only at $\alpha = 13°$ for the NASA Trap-Wing, and $\alpha = 0°$ for the DLR-F6.

Table 11 compares the performance of the *a*-FMG algorithm with LBE smoother with the implicit (single-order) time integration for both aircraft. The FMG algorithm allows to reduce the computational time up to $\sim 70\%$ for the NASA Trap-Wing, and up to $\sim 60\%$ for the DLR-F6. The performance of the FMG algorithm for the NASA Trap-Wing decreases for higher solution approximations, while the reduction of the computational cost for the DLR-F6 is almost constant for all solution approximations.

9. Conclusion

In this work, a three-dimensional *p*-MG algorithm based on nonlinear FAS and FMG is improved and assessed for the high-order dG approximation of the RANS and the k-$\widetilde{\omega}$ turbulence model equations. The reduction of the computational time with respect to the implicit (single-order) time integration is demonstrated with different smoothers, e.g., BIRK, LIRK, and LBE, on different benchmark, verification, and industrial cases, representative of different flow conditions.

The main findings of this work are:

(*i*) the algorithm does not show an asymptotic scalability, or the solution approximation independence condition, i.e., the number of iterations to reach the convergence for each solution approximation is different (see Secs. 7 and 8),

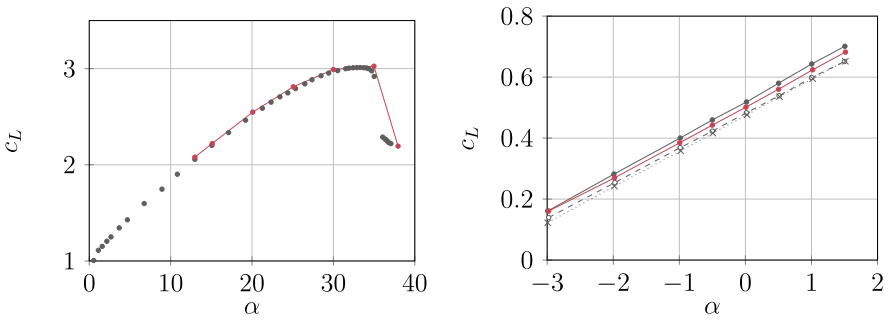


Fig. 15. NASA's workshop aircraft. Lift coefficient, c_L, with different angles of attack for the NASA Trap-Wing (left) and the DLR-F6 (right). Simulations are carried out at $\alpha = \{13, 15, 20, 25, 30, 35, 38°\}$ for the NASA Trap-Wing and $\alpha = \{-3, -2, -1, -0.5, 0, 0.5, 1, 1.5°\}$ for the DLR-F6. The results of the 3rd AIAA CFD Drag Prediction Workshop (DPW3) [46] are with three different meshes with 2.7, 8, and 24×10^6 elements. • HiLift-1 exp. [45], —●— DPW3 k-ω [46], —●— $\mathbb{P}^3$ solution.

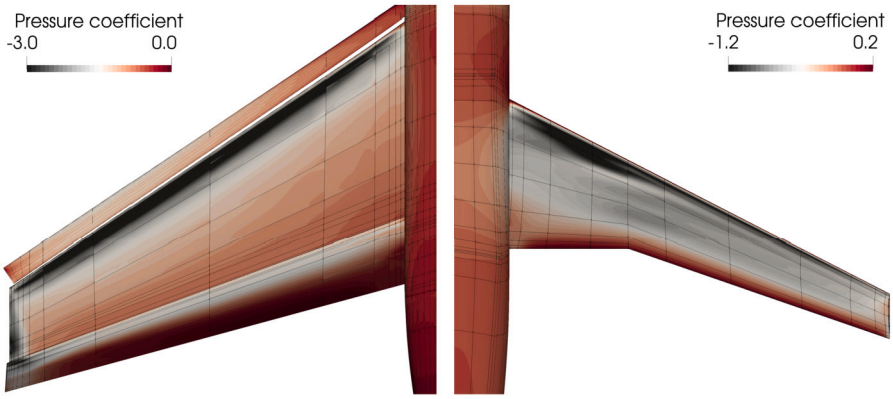


Fig. 16. NASA's workshop aircraft. Pressure coefficient, c_p, distribution on the fuselage and wing for the NASA Trap-Wing (left) and the DLR-F6 (right), $\mathbb{P}^3$ solution.

Table 11
NASA's workshop aircraft. Performance of the a-FMG algorithm with LBE smoother for the NASA Trap-Wing (top) and DLR-F6 (bottom) model, $tol_{GMRES} = 10^{-12}$, $CFL_{min} = 1$, $\mathbb{P}^{coarsest} = \mathbb{P}^0$. $\mathbb{P}^l$ is the finest level, ν_{pre} and ν_{post} the number of pre- and post-smoothing iterations for intermediate and finest levels, N_{ite} the number of iterations to reach the convergence, t_{CPU} the computational time, and Δt_{CPU} the reduction due to the use of the FMG algorithm in comparison to the traditional implicit time integration.

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	19010	1.13×10^4	-
	a-FMG	1	3	11906	2.89×10^3	−74.49%
2	SO	-	-	53228	7.69×10^4	-
	a-FMG	1	2	11925	2.32×10^4	−69.82%
3	SO	-	-	64938	3.96×10^5	-
	a-FMG	1	1	11962	1.33×10^5	−64.01%

$\mathbb{P}^l$		ν_{pre}	ν_{post}	N_{ite}	t_{CPU}	Δt_{CPU}
1	SO	-	-	1222	9.05×10^3	-
	a-FMG	1	3	648	3.11×10^3	−62.59%
2	SO	-	-	1483	6.15×10^4	-
	a-FMG	1	2	680	2.40×10^4	−61.06%
3	SO	-	-	1745	2.95×10^5	-
	a-FMG	1	1	743	1.02×10^5	−65.55%

(*ii*) the BIRK/LIRK smoothers show better performance in the validation cases in comparison to the LBE smoother, as a higher reduction of the computational cost and memory footprint is observed (see Sec. 6),

(*iii*) the higher performance of the BIRK/LIRK smoothers is observed only if $\mathbb{P}^1$ is considered as the coarsest level, while the performance deteriorates if the simulation is initialized with freestream conditions, i.e., the $\mathbb{P}^0$ acts as the coarsest level (see Sec. 6),

(*iv*) the LBE smoother enhances the computational time regardless of the chosen coarsest level; the results for all considered test cases show a reduction of the computational time between 50% and 95% with respect to the implicit (single-order) time integration (see Secs. 6, 7, and 8),

(*v*) the automatic choice of the number of the pre- and post-smoothing iterations, ν_{pre} and ν_{post}, provides a computational time comparable, or lower than the value obtained with manually optimized smoothing iterations. Moreover, the number of smoothing iterations can be different for each level, providing a further reduction of the computing time (see Secs. 6, 7, and 8),

(*vi*) the performance of the algorithm decreases for higher solution approximations in the subsonic test cases, while the computing time reduction is almost p independent in transonic test cases (see Secs. 7.2, and 8.1),

(*vii*) the AR of the mesh elements, in the boundary layer, in shock and wake regions, does not seem to influence the performance of the algorithm (see Secs. 6.1, and 7.2),

The algorithm is proposed for the RANS equations without any ad-hoc modifications for the treatment of the variables of the k-$\widetilde{\omega}$ turbulence model. This general formulation allows an easy coupling with other turbulence and transition models for steady-state simulations. To enhance the computational efficiency of the proposed solver, the use of the p-MG as preconditioner will be investigated, considering the low memory smoothers LIRK/BIRK and a MF approach. Moreover, agglomeration-based h- and p-MG will be compared to assess their performance and investigate a favorable strategy to exploit their coupling. Even if some parts of the h-MG still need to be implemented, the use of basis functions defined in the real space and the integration over the mesh elements to obtain high-order Gaussian quadrature rules adopted in the actual framework will simplify h-MG implementation.

CRediT authorship contribution statement

D. Bulgarini: Visualization, Methodology. **A. Ghidoni:** Writing – original draft, Supervision, Project administration, Conceptualization. **E. Mantecca:** Visualization, Methodology. **G. Noventa:** Writing – original draft, Visualization, Validation, Investigation, Formal analysis, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors sincerely thank Prof. Alessandro Colombo of Università degli Studi di Bergamo, who provided the meshes used for the simulations of the DrivAer car.

Data availability

Data will be made available on request.

References

[1] T.C. Warburton, I. Lomtev, Y. Du, S. Sherwin, G.E. Karniadakis, Galerkin and discontinuous Galerkin spectral/hp methods, Comput. Methods Appl. Mech. Eng. 175 (1999) 343–359.

[2] F. Bassi, N. Franchina, A. Ghidoni, S. Rebay, A numerical investigation of a spectral-type nodal collocation discontinuous Galerkin approximation of the Euler and Navier–Stokes equations, Int. J. Numer. Methods Fluids 71 (10) (2013) 1322–1339, https://doi.org/10.1002/fld.3713.

[3] F. Massa, G. Noventa, M. Lorini, F. Bassi, A. Ghidoni, High-order linearly implicit two-step peer schemes for the discontinuous Galerkin solution of the incompressible Navier–Stokes equations, Comput. Fluids 162 (2018) 55–71, https://doi.org/10.1016/j.compfluid.2017.12.003.

[4] G. Noventa, F. Massa, S. Rebay, F. Bassi, A. Ghidoni, Robustness and efficiency of an implicit time-adaptive discontinuous Galerkin solver for unsteady flows, Comput. Fluids 204 (2020), https://doi.org/10.1016/j.compfluid.2020.104529.

[5] U. Trottenberg, C.W. Oosterlee, A. Schuller, Multigrid, Academic Press, 525 B Street, Suite 1900, San Diego, California 92101-4495, USA, 2001.

[6] A. Colombo, A. Ghidoni, G. Noventa, S. Rebay, *p*-Multigrid high-order discontinuous Galerkin solution of compressible flows, in: M. Kronbichler, P.-O. Persson (Eds.), CISM International Centre for Mechanical Sciences, Efficient High-Order Discretizations for Computational Fluid Dynamics, Springer Science, 2021.

[7] A. Jameson, Solution of the Euler equations for two dimensional transonic flow by a multigrid method, Appl. Math. Comput. 13 (3) (1983) 327–355, https://doi.org/10.1016/0096-3003(83)90019-X, https://www.sciencedirect.com/science/article/pii/009630038390019X.

[8] G. Carré, An implicit multigrid method by agglomeration applied to turbulent flows, Comput. Fluids 26 (3) (1997) 299–320, https://doi.org/10.1016/S0045-7930(96)00045-X, https://www.sciencedirect.com/science/article/pii/S004579309600045X.

[9] D.J. Mavriplis, An assessment of linear versus nonlinear multigrid methods for unstructured mesh solvers, J. Comput. Phys. 175 (1) (2002) 302–325, https://doi.org/10.1006/jcph.2001.6948.

[10] K.J. Fidkowski, T.A. Oliver, J. Lu, L. Darmofal, *p*-Multigrid solution of high-order discontinuous Galerkin discretizations of the compressible Navier-Stokes equations, J. Comput. Phys. 207 (1) (2005) 92–113.

[11] K. Shahbazi, D.J. Mavriplis, N.K. Burgess, Multigrid algorithms for high-order discontinuous Galerkin discretizations of the compressible Navier–Stokes equations, J. Comput. Phys. 228 (21) (2009) 7917–7940, https://doi.org/10.1016/j.jcp.2009.07.013.
[12] P.-O. Persson, J. Peraire, Newton-GMRES preconditioning for discontinuous Galerkin discretizations of the Navier–Stokes equations, SIAM J. Sci. Comput. 30 (6) (2008) 2709–2733, https://doi.org/10.1137/070692108.
[13] L.T. Diosady, D.L. Darmofal, Preconditioning methods for discontinuous Galerkin solutions of the Navier–Stokes equations, J. Comput. Phys. 228 (11) (2009) 3917–3935, https://doi.org/10.1016/j.jcp.2009.02.035.
[14] J. Gopalakrishnan, G. Kanschat, A multilevel discontinuous Galerkin method, Numer. Math. 95 (2003) 527–550, https://doi.org/10.1007/s002110200392.
[15] P. Bastian, M. Blatt, R. Scheichl, Algebraic multigrid for discontinuous Galerkin discretizations of heterogeneous elliptic problems, Numer. Linear Algebra Appl. 19 (2) (2012) 367–388, https://doi.org/10.1002/nla.1816.
[16] P.F. Antonietti, L. Melas, Algebraic multigrid schemes for high-order nodal discontinuous Galerkin methods, SIAM J. Sci. Comput. 42 (2) (2020) A1147–A1173, https://doi.org/10.1137/18M1204383.
[17] P.F. Antonietti, M. Sarti, M. Verani, Multigrid algorithms for *hp*-discontinuous Galerkin discretizations of elliptic problems, SIAM J. Numer. Anal. 53 (1) (2015) 598–618, https://doi.org/10.1137/130947015.
[18] B.T. Helenbrook, D.J. Mavriplis, H.A. Atkins, Analysis of *p*-multigrid for continuous and discontinuous finite element discretizations, in: 16th AIAA Computational Fluid Dynamics Conference, AIAA-2003, AIAA, Orlando, Florida, 2003.
[19] D. Darmofal, K. Fidkowski, Development of a higher-order solver for aerodynamic applications, in: 42nd AIAA Aerospace Sciences Meeting and Exhibit, AIAA 2004-112, AIAA, 2004.
[20] F. Bassi, N. Franchina, A. Ghidoni, S. Rebay, Spectral *p*-multigrid discontinuous Galerkin solution of the Navier-Stokes equations, Int. J. Numer. Methods Fluids 67 (11) (2011) 1540–1558, https://doi.org/10.1002/fld.2430.
[21] A. Ghidoni, A. Colombo, F. Bassi, S. Rebay, Efficient *p*-multigrid discontinuous Galerkin solver for complex viscous flows on stretched grids, Int. J. Numer. Methods Fluids 75 (2) (2014) 134–154, https://doi.org/10.1002/fld.3888.
[22] A.M. Rueda-Ramírez, J. Manzanero, E. Ferrer, G. Rubio, E. Valero, A *p*-multigrid strategy with anisotropic *p*-adaptation based on truncation errors for high-order discontinuous Galerkin methods, J. Comput. Phys. 378 (2019) 209–233, https://doi.org/10.1016/j.jcp.2018.11.009.
[23] A. Ghidoni, D. Pasquale, S. Rebay, A. Colombo, F. Bassi, *p*-Multigrid discontinuous Galerkin method for compressible turbulent flows, in: 51st AIAA Aerospace Sciences Meeting Including the New Horizons Forum and Aerospace Exposition 2013, AIAA 2013-1002, AIAA, 2007.
[24] M. Wallraff, T. Leicht, M. Lange-Hegermann, Numerical flux functions for Reynolds-averaged Navier–Stokes and k-ω turbulence model computations with a line-preconditioned *p*-multigrid discontinuous Galerkin solver, Int. J. Numer. Methods Fluids 71 (8) (2013) 1055–1072, https://doi.org/10.1002/fld.3702.
[25] Z. Jiang, C. Yan, J. Yu, W. Yuan, Practical aspects of *p*-multigrid discontinuous Galerkin solver for steady and unsteady RANS simulations, Int. J. Numer. Methods Fluids 78 (11) (2015) 670–690, https://doi.org/10.1002/fld.4035.
[26] W.A. Mulder, B. Van Leer, Experiments with implicit upwind methods for the Euler equations, J. Comput. Phys. 59 (2) (1985) 232–246, https://doi.org/10.1016/0021-9991(85)90144-5.
[27] F. Bassi, L. Botti, A. Colombo, A. Crivellini, N. Franchina, A. Ghidoni, Assessment of a high-order accurate discontinuous Galerkin method for turbomachinery flows, Int. J. Comput. Fluid Dyn. 30 (4) (2016) 307–328, https://doi.org/10.1080/10618562.2016.1198783.
[28] D. Knoll, D. Keyes, Jacobian-free Newton–Krylov methods: a survey of approaches and applications, J. Comput. Phys. 193 (2) (2004) 357–397, https://doi.org/10.1016/j.jcp.2003.08.010.
[29] A. Crivellini, F. Bassi, An implicit matrix-free discontinuous Galerkin solver for viscous and turbulent aerodynamic simulations, Comput. Fluids 50 (1) (2011) 81–93, https://doi.org/10.1016/j.compfluid.2011.06.020.
[30] F. Bassi, A. Crivellini, S. Rebay, M. Savini, Discontinuous Galerkin solution of the Reynolds-averaged Navier-Stokes and k-ω turbulence model equations, Comput. Fluids 34 (4) (2005) 507–540, https://doi.org/10.1016/j.compfluid.2003.08.004, residual Distribution Schemes, Discontinuous Galerkin Schemes and Adaptation.
[31] S. Balay, K. Buschelman, W.D. Gropp, D. Kaushik, M.G. Knepley, L.C. McInnes, B.F. Smith, H. Zhang, PETSc web page, http://www.mcs.anl.gov/petsc, 2001.
[32] A. Ghidoni, A. Colombo, F. Bassi, S. Rebay, Efficient *p*-multigrid discontinuous Galerkin solver for complex viscous flows on stretched grids, Int. J. Numer. Methods Fluids 75 (2) (2014) 134–154, https://doi.org/10.1002/fld.3888.
[33] L. Thomas, Elliptic problems in linear differential equations over a network, Tech. Rep. Watson Sci. Comput. Lab Report, Columbia University, New York, 1949.
[34] The second international Vortex Flow Experiment (VFE-2), http://vfe2.dlr.de.
[35] G. Falcone, H. Kredel, M. Krietemeyer, D. Merten, M. Merz, F.-J. Pfreundt, C. Simmendinger, D. Versick, Integrated performance analysis of computer systems (ipacs). Benchmarks for distributed computer systems, Prax. Inf.verarb. Kommun. 28 (3) (2005) 150–159, https://doi.org/10.1515/PIKO.2005.150 [cited 2024-11-11].
[36] S. Crippa, Advances in vortical flow prediction methods for design of delta-winged aircraft, 2008, no. 2008:30 in Trita-AVE, KTH, viii, 38, qC 20100713.
[37] J.C. Vogel, J.K. Eaton, Combined heat transfer and fluid dynamic measurements downstream of a backward-facing step, J. Heat Transf. 107 (4) (1985) 922–929, https://doi.org/10.1115/1.3247522.
[38] RAE2822 transonic airfoil, https://www.grc.nasa.gov/www/wind/valid/raetaf/raetaf.html. (Accessed 5 October 2022).
[39] 2dbfs: 2d backward facing step, https://turbmodels.larc.nasa.gov/backstep_val.html. (Accessed 20 August 2022).
[40] D.M. Driver, H.L. Seegmiller, Features of a reattaching turbulent shear layer in divergent channelflow, AIAA J. 23 (2) (1985) 163–171, https://doi.org/10.2514/3.8890.
[41] P.R. Spalart, S. Deck, M.L. Shur, K.D. Squires, M.K. Strelets, A. Travin, A new version of detached-eddy simulation, resistant to ambiguous grid densities, Theor. Comput. Fluid Dyn. 20 (181) (2006) 181–195, https://doi.org/10.1007/s00162-006-0015-0.
[42] A. Travin, M. Shur, M. Strelets, P.R. Spalart, Physical and numerical upgrades in the detached-eddy simulation of complex turbulent flows, in: R. Friedrich, W. Rodi (Eds.), Advances in LES of Complex Flows, Springer, Netherlands, 2002, pp. 239–254.
[43] P. Cook, M. McDonald, M. Firmin, Aerofoil RAE2822 - pressure distributions, and boundary layer and wake measurements, Experimental Data Base for Computer Program Assessment, AGARD Report AR 138, 1979.
[44] A.I. Heft, T. Indinger, N.A. Adams, Experimental and numerical investigation of the DrivAer model, in: Symposia, Parts A and B of Fluids Engineering Division Summer Meeting, vol. 1, 2012, pp. 41–51.
[45] 1st AIAA cfd high lift prediction workshop, http://hiliftpw.larc.nasa.gov/index-workshop1.html. (Accessed 30 September 2023).
[46] 3rd AIAA cfd drag prediction workshop, http://aiaa-dpw.larc.nasa.gov/Workshop3/workshop3.html. (Accessed 30 September 2023).
[47] H. Hoheisel, Entwicklung neuer Entwurfskonzepte fur zwei Turbinengitter, Teil III, Ergebnisse T106, Technical report, Institut fur Entwurfsaerodynamik, Braunschweig, 1981.
[48] M. Lorini, F. Bassi, A. Colombo, A. Ghidoni, G. Noventa, Discontinuous Galerkin solution of the RANS and k_l-k-log(ω) equations for natural and bypass transition, Comput. Fluids 214 (2021) 104767, https://doi.org/10.1016/j.compfluid.2020.104767.
[49] F. Bassi, A. Colombo, A. Ghidoni, M. Lorini, G. Noventa, Discontinuous Galerkin solution of the Reynolds-averaged Navier-Stokes and $k_L - k_T - \omega$ transition model equations, in: ECCOMAS Congress 2016 - Proceedings of the 7th European Congress on Computational Methods in Applied Sciences and Engineering, vol. 2, 2016, pp. 2632–2647.
[50] M. Rüttgers, J. Park, D. You, Large-eddy simulation of turbulent flow over the DrivAer fastback vehicle model, J. Wind Eng. Ind. Aerodyn. 186 (2019) 123–138, https://doi.org/10.1016/j.jweia.2019.01.003.

[51] A. Colombo, A. Bortoli, P. Conti, A. Crivellini, A. Ghidoni, A. Nigro, G. Noventa, Assessment of a discontinuous Galerkin method for the simulation of the turbulent flow around the DrivAer car model, Appl. Sci. 11 (21) (2021), https://doi.org/10.3390/app112110202.
[52] N. Ashton, A. West, S. Lardeau, A. Revell, Assessment of RANS and DES methods for realistic automotive models, Comput. Fluids 128 (2016) 1–15, https://doi.org/10.1016/j.compfluid.2016.01.008.
[53] Gridpro website, https://www.gridpro.com/. (Accessed 9 December 2019) [Online].